

SAPIENZA
UNIVERSITÀ DI ROMA

Modellazione MIP

Modellazione

La parte metodologica del corso: che cos'è un modello MIP, logica e variabili binarie, i quattordici legami fra variabili con le loro dimostrazioni, rilassamenti, dualità e bound, euristiche costruttive, e il passaggio dal modello a Python/Gurobi. I problemi su cui applicare tutto questo stanno nelle dispense dei problemi numerici e dei problemi con modello simbolico.

Fabio Furini

fabiofurini.github.io

Dipartimento di Ingegneria informatica, automatica e gestionale — Sapienza Università di Roma

Versione online e materiale scaricabile:
fabiofurini.github.io/modellazione-mip/
github.com/fabiofurini/modellazione-mip

Anno accademico 2026–2027

Indice

1 Che cos'è un modello MIP **3**

1.1 Dati, variabili, obiettivo, vincoli . . . 3
1.2 Perché l'interezza conta . . . 6
1.3 Il rilassamento LP e il sandwich dei bound . . . 8
1.4 Tre pattern che tornano ovunque . . . 10
1.5 Che cosa fa il solver con i due bound . . . 10
1.6 Quello che questo capitolo ha messo in mano . . . 10

2 Rilassamenti, dualità e bound **12**

2.1 Che cos'è un rilassamento . . . 12
2.2 La tabella di conversione primale/duale . . . 13
2.3 Dualità debole, dualità forte . . . 13
2.4 Tre ricette per costruire a mano una soluzione duale . . . 14
2.5 Un problema di minimo, per esteso . . . 15
2.6 Un problema di massimo: i ruoli si scambiano . . . 17
2.7 Disuguaglianze valide e vincoli che preservano l'ottimalità . . . 20
2.8 Formulazioni più forti: che cosa vuol dire, e come si dimostra . . . 21
2.9 Quello che dice il solver . . . 21
2.10 I duali dell'LP non sono i prezzi marginali del MILP . . . 22
2.11 Il protocollo dei bound . . . 22

3 Dal modello a Python/Gurobi **24**

3.1 Le istruzioni di base di gurobipy . . . 24
3.2 Le quattro classi di variabili . . . 26
3.3 Il modello, una famiglia per blocco . . . 26
3.4 Leggere i risultati . . . 27
3.5 Le tolleranze . . . 28
3.6 Il rilassamento con `relax()` . . . 28
3.7 Tre problemi che il corso riusa . . . 29
3.8 Il protocollo del corso, dall'inizio alla fine . . . 34
3.9 Eseguire il codice . . . 35

4 Euristiche costruttive **37**
4.1 Bin packing: le regole di inserimento . . . 38
4.2 $P||C_{\max}$: la regola del meno carico . . . 39
4.3 Set covering: la regola del completamento più economico . . . 40
4.4 Zaino: la regola del rapporto migliore . . . 41
4.5 TSP: il vicino più vicino . . . 42
4.6 Lot sizing: copertura di periodi a costo unitario minimo . . . 43
4.7 Ricerca locale, e che cosa non dà . . . 44
4.8 Quando un'euristica costruttiva fallisce . . . 44
4.9 Che cosa dà, e che cosa non dà . . . 45
4.10 Il quadro delle euristiche . . . 46

5 Logica e variabili binarie **48**
5.1 Proposizioni e variabili binarie . . . 48
5.2 Letterali, clausole, forma normale congiuntiva . . . 49
5.3 Da CNF a vincoli lineari . . . 50
5.4 Implicazioni logiche . . . 51
5.5 Scindere un'implicazione: quando si può . . . 53
5.6 Contare: al più uno, almeno uno, esattamente uno . . . 53
5.7 Operatori logici e operatori insiemistici . . . 54
5.8 Verificare la traduzione, non fidarsene . . . 54
5.9 Cinque esercizi . . . 54
5.10 I vincoli logici dentro un modello di ottimizzazione . . . 56
5.11 Quello che questo capitolo ha messo in mano . . . 59

6 Legami fra variabili **60**
6.1 Attivazione: forma aggregata e forma disaggregata . . . 61
6.2 Costo fisso, capacità e flusso continuo . . . 62
6.3 Lotto minimo e variabile semicontinua . . . 63
6.4 Conteggi interi, capacità multipla, arrotondamento all'intero superiore . . . 64
6.5 L'ausiliaria di massimo . . . 64
6.6 Min-max, max-min e differenza fra il massimo e il minimo . . . 65
6.7 Il valore assoluto . . . 66
6.8 Big-M: vincoli condizionati e disgiunzioni . . . 67
6.9 Precedenze e sequenziamento . . . 68
6.10 «Se e solo se» . . . 69
6.11 Contare i tipi diversi . . . 70
6.12 Alldiff ed espansione binaria . . . 70
6.13 Vincoli violabili, deviazioni e penalità . . . 71
6.14 Funzioni lineari a tratti . . . 72
6.15 La mappa delle tecniche . . . 74

Indice analitico **76**

Capitolo 1

Che cos'è un modello MIP

Script: `python/cap01_modelli.py` Pagina online

1.1 Dati, variabili, obiettivo, vincoli

Un modello di *programmazione matematica* traduce una decisione in quattro ingredienti, sempre nello stesso ordine:

1. i **dati**: i numeri che descrivono il problema, noti prima di decidere (costi, capacità, domande, tempi...);
2. le **variabili**: le quantità che il decisore controlla;
3. l'**obiettivo**: una funzione delle variabili da minimizzare o massimizzare;
4. i **vincoli**: le equazioni e disequazioni che le variabili devono soddisfare perché la soluzione sia realizzabile.

Un problema di *ottimizzazione* chiede la soluzione ammissibile (che soddisfa tutti i vincoli) di valore migliore per l'obiettivo.

Le classi di modelli

- **Le sigle**: LP dritto è la *classe* di problemi, *LP* in corsivo è *un* problema di quella classe — quello di cui si scrivono $\tilde{x}(LP)$ e $z(LP)$. Lo stesso per ILP, BIP e MILP.
- **La taglia e i dati**: n variabili, con $j \in \{1, 2, \ldots, n\}$, e m vincoli, con $i \in \{1, 2, \ldots, m\}$; i costi c_j, i coefficienti a_{ij} e i termini noti b_i.

Che cosa può contenere un modello

- una **funzione obiettivo lineare** nelle variabili, da minimizzare o da massimizzare;
- **vincoli lineari** di tre tipi: di verso $\geq$, per una richiesta da soddisfare; di verso $\leq$, per una capacità da rispettare; di uguaglianza, per un bilancio o un'assegnazione esatta;
- **variabili**, in una o più famiglie, ciascuna con il proprio dominio, dichiarato in fondo al modello.

Nessuna delle tre cose ha una forma obbligata: quanti vincoli, di quale tipo e con quante famiglie di variabili lo decide il problema.

Che cosa distingue le quattro classi: i domini

- **LP**: tutte le variabili sono continue, $x_j \in \mathbb{R}$, di solito non negative. In questo corso un modello di LP non si scrive mai da zero: arriva come *rilassamento* di un modello MIP, quando si lasciano cadere i vincoli di interezza.
- **ILP**: tutte le variabili sono intere, $x_j \in \mathbb{Z}$.
- **BIP**: tutte le variabili sono binarie, $x_j \in \{0, 1\}$.
- **MILP**: alcune variabili sono intere o binarie e altre continue.

È lì che vive la differenza di difficoltà: un LP si risolve in tempo polinomiale, un ILP e un MILP in generale no.

Un modello di ILP

Quante unità comprare da due fornitori, x_1 e x_2, a lotti interi.

$$\begin{aligned} \min \quad & 4x_1 + 7x_2 \\ \text{soggetto a} \quad & 2x_1 + 3x_2 \geq 12 \\ & x_1 + x_2 \leq 5 \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

L'obiettivo minimizza la spesa. Il primo vincolo, di verso $\geq$, impone di coprire la domanda di 12 unità; il secondo, di verso $\leq$, limita a cinque i lotti totali. L'ultima riga è il dominio: tutte le variabili sono intere, e il modello è un ILP.

Un modello di BIP

Quali progetti finanziare: $y_k = 1$ se il progetto k si finanzia, 0 altrimenti.

$$\begin{aligned} \max \quad & 5y_1 + 4y_2 + 6y_3 \\ \text{soggetto a} \quad & y_1 + y_2 + y_3 \leq 2 \\ & y_1 + y_3 \geq 1 \\ & y_1, y_2, y_3 \in \{0, 1\} \end{aligned}$$

L'obiettivo massimizza il valore dei progetti scelti. Il primo vincolo, di verso $\leq$, dice che se ne possono finanziare al più due; il secondo, di verso $\geq$, che almeno uno fra il primo e il terzo va finanziato. L'ultima riga è il dominio: tutte le variabili sono binarie, e il modello è un BIP.

Un modello di MILP

Due prodotti da fabbricare, con x_1 e x_2 le quantità prodotte, e un impianto da attivare o no, con $y = 1$ se lo si attiva e $y = 0$ altrimenti.

$$\begin{array}{rrcrcrcl} \max & 3x_1 & + & 8x_2 & - & 10y & & \\ \text{soggetto a} & x_1 & + & x_2 & & & = & 6 \\ & & & x_2 & - & 4y & \leq & 0 \\ & x_1 & & & & & \geq & 1 \\ & x_1, & & x_2 & & & \geq & 0 \\ & & & & & y & \in & \{0,1\} \end{array}$$

L'obiettivo massimizza il ricavo dei due prodotti meno il costo dell'impianto. Il primo vincolo, di uguaglianza, impone di produrre esattamente le sei unità richieste; il secondo, di verso $\leq$, dice che del secondo prodotto se ne possono fare al più quattro unità e soltanto a impianto attivo, perché con $y = 0$ resta $x_2 \leq 0$; il terzo, di verso $\geq$, chiede almeno un'unità del primo prodotto. Le ultime due righe sono i domini: due variabili continue e una binaria, e il modello è un MILP.

Ogni dato, ogni insieme e ogni simbolo è *definito prima di essere usato*. In ogni modello della dispensa:

1. le variabili sono introdotte esplicitamente prima della formulazione;
2. obiettivo e ciascuna famiglia di vincoli hanno un numero d'equazione;
3. i vincoli di dominio, che dichiarano le variabili, chiudono il modello;
4. dopo il modello un elenco spiega obiettivo e vincoli famiglia per famiglia.

Notazione dei valori ottimi e dei bound

- I **valori ottimi** si scrivono indicando sempre di quale modello si parla: $z(MILP)$ è l'ottimo di un modello di MILP, e allo stesso modo si scriveranno quelli dei rilassamenti e dei duali che il capitolo introduce più avanti. Il modello che si sta ottimizzando è sempre esplicito nel simbolo.
- Le soluzioni **ammissibili** portano la barra — comunque siano state costruite — e quelle **ottime** la tilde:

$$(\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n) \qquad \text{e} \qquad (\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_n).$$

- I **bound** si chiamano LB e UB:

$$LB \;\leq\; z(MILP) \;\leq\; UB.$$

  Quando serve dire da quale soluzione arriva un bound, lo si scrive fra parentesi: una

soluzione ammissibile $(\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$ di un modello di minimo dà

$$UB(\bar{x}) = \sum_{j=1}^{n} c_j \, \bar{x}_j,$$

e in un modello di massimo la stessa somma è $LB(\bar{x})$. Quale dei due venga da una soluzione ammissibile e quale da un rilassamento dipende dunque dal verso dell'obiettivo: in un problema di minimo una soluzione ammissibile dà UB e il rilassamento dà LB, in un problema di massimo è il contrario.

Questo corso lavora quasi solo con MILP: variabili binarie o intere per le decisioni «sì/no» o di conteggio, variabili continue per quantità che si misurano (tempo, denaro, flusso).

1.2 Perché l'interezza conta

Un ILP o un MILP non si risolve «arrotondando» la soluzione del suo rilassamento continuo. Un controesempio minimo lo mostra.

L'arrotondamento fallisce

$$\begin{array}{rrcrcl} \max & x_1 & + & x_2 & & \\ \text{soggetto a} & 2x_1 & + & 2x_2 & \leq & 3 \\ & x_1, & & x_2 & \in & \{0,1\} \end{array} \tag{1.1}$$

Il rilassamento LP sostituisce il dominio binario con

$$0 \leq x_1 \leq 1, \qquad 0 \leq x_2 \leq 1,$$

e ha valore ottimo

$$z(LP^+) = \frac{3}{2}.$$

Su questo valore vanno notate tre cose.

- È raggiunto da *infinite* soluzioni ottime: tutti i punti del segmento

  $$x_1 + x_2 = \frac{3}{2}, \qquad 0 \leq x_1 \leq 1, \qquad 0 \leq x_2 \leq 1,$$

  fra cui (3/4, 3/4), (1, 1/2) e (1/2, 1).
- Quale di queste restituisca il solver dipende dall'algoritmo, non dal modello: nessuna è «la» soluzione del rilassamento. Sulla nostra installazione Gurobi restituisce il vertice (1/2, 1).
- Arrotondando si ottengono esiti diversi a seconda del punto di partenza *e* del verso:
  - da (3/4, 3/4), per eccesso si ottiene (1, 1), che viola il vincolo perché $2 + 2 = 4 > 3$; per difetto si ottiene (0, 0), ammissibile ma di valore 0. Da questo punto *nessuno* dei due versi trova l'ottimo;

  - da $(1,\ 1/2)$, per eccesso si ottiene ancora $(1,\ 1)$, non ammissibile; per difetto si ottiene $(1,\ 0)$, ammissibile e di valore 1, che è proprio l'ottimo intero $z(MILP) = 1$.

  Non c'è dunque un verso «giusto»: su questa istanza funziona il difetto, e solo a partire da uno dei due punti. Nulla, prima di risolvere, diceva quale dei due il solver avrebbe restituito.

Le lezioni sono due, distinte.

- L'arrotondamento può produrre punti *non ammissibili*, e quando ne produce di ammissibili non c'è alcuna garanzia sul loro valore.
- Il divario
$$\frac{3}{2} - 1 = \frac{1}{2}$$
non è colpa dell'arrotondamento: *nessun* punto intero ammissibile vale più di 1.

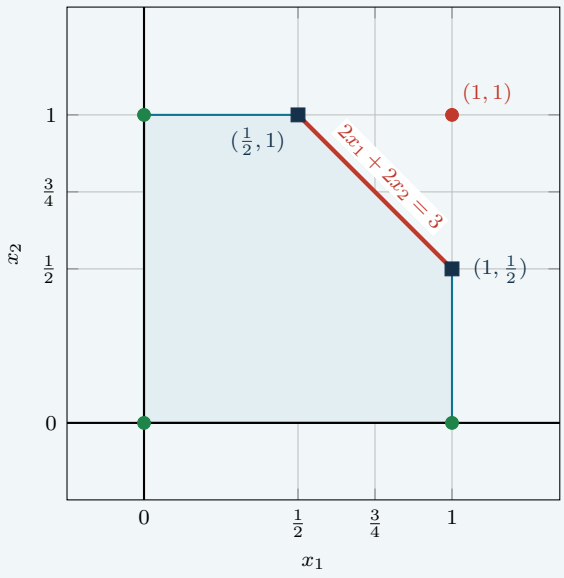


Figura 1.1: L'esempio (1.1). In azzurro la regione ammissibile del rilassamento LP^+; in verde i tre punti interi ammissibili, in rosso $(1, 1)$ che viola il vincolo. I due quadrati blu sono due delle infinite soluzioni ottime del rilassamento, tutte di valore 3/2.

La lezione: il rilassamento continuo dà un'indicazione (un bound, si veda sotto), non una soluzione. I MILP si risolvono con algoritmi dedicati, non con euristiche ingenue sul continuo.

1.3 Il rilassamento LP e il sandwich dei bound

Rilassamento LP: due versioni da non confondere

Il **rilassamento LP** di un MILP si ottiene togliendo i vincoli di interezza e binarietà e tenendo tutto il resto. Ci sono due modi di farlo, e il corso li usa entrambi:

- il **rilassamento con i bound**, $z(LP^+)$: $x \in \{0,1\}$ diventa $0 \le x \le 1$, $x \in \mathbb{Z}_{\ge 0}$ diventa $x \ge 0$. È quello che risolve Gurobi con `relax()` ed è il primo rilassamento che il solver risolve;
- il **rilassamento senza i bound**, $z(LP)$: $x \in \{0,1\}$ diventa il solo $x \ge 0$, senza riportare il limite superiore. È più debole (o uguale) del precedente, ma ha un duale più compatto: è di questo che negli esercizi si scrive il duale a mano.

Il rilassamento senza i bound è il più debole dei due, cioè il bound *più lontano* dall'ottimo intero. I due coincidono quando gli altri vincoli del modello implicano già $x \le 1$: succede, per esempio, quando c'è un vincolo di assegnamento $\sum_m x_{jm} = 1$. Da che parte stiano rispetto a $z(MILP)$ lo dice il riquadro qui sotto.

Il rilassamento è un bound: da che parte, e perché

Il rilassamento *toglie* vincoli, quindi ogni soluzione ammissibile del MILP resta ammissibile per il rilassamento, mentre il viceversa è falso: il rilassamento ottimizza su un insieme di soluzioni più grande. Ottimizzare su un insieme più grande non può dare un valore peggiore, quindi:

$$\text{massimo:} \quad z(LP) \ge z(LP^+) \ge z(MILP);$$

$$\text{minimo:} \quad z(LP) \le z(LP^+) \le z(MILP).$$

In un massimo il rilassamento è un *upper* bound, in un minimo un *lower* bound. In entrambi i casi è un bound **ottimistico**: promette almeno quanto l'ottimo vero, e in generale di più. Non è mai una soluzione, e il valore promesso può essere irraggiungibile.

La regola è questa, e vale in tutto il corso.

- I **bound primali** vengono dalle *soluzioni intere ammissibili*: una soluzione che esiste davvero, costruita a mano o trovata dal solver. Sono il lato pessimistico.
- I **bound duali** vengono o dalla *soluzione ottima del rilassamento LP*, o da una qualunque *soluzione ammissibile del suo duale*. Sono il lato ottimistico.

Il duale del rilassamento LP, dunque, *non* dà un bound «dall'altro lato»: per dualità debole, in un problema di minimo ogni soluzione duale ammissibile vale al più $z(LP)$, quindi al più $z(MILP)$, e sta dalla *stessa* parte del rilassamento — in generale più debole. Riassumendo: sia $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ una soluzione ammissibile del MILP, di valore $\sum_{j=1}^{n} c_j \bar{x}_j$, e $(\bar{\pi}_1, \bar{\pi}_2, \dots, \bar{\pi}_m)$ una soluzione ammissibile del duale del rilassamento, di valore $\sum_{i=1}^{m} b_i \bar{\pi}_i$. In un problema di **minimo** la prima dà $UB(\bar{x})$ e la seconda $LB(\bar{\pi})$:

$$LB(\bar{\pi}) \;\le\; z(D(LP)) = z(LP) \;\le\; z(LP^+) \;\le\; z(MILP) \;\le\; UB(\bar{x}),$$

in un problema di **massimo** i ruoli si scambiano, la prima dà $LB(\bar{x})$ e la seconda $UB(\bar{\pi})$:

$$LB(\bar{x}) \;\le\; z(MILP) \;\le\; z(LP^{+}) \;\le\; z(LP) = z(D(LP)) \;\le\; UB(\bar{\pi}).$$

L'uguaglianza in mezzo è la dualità forte: il duale del rilassamento vale quanto il rilassamento, e la disuguaglianza vera è quella fra il *valore di una soluzione duale ammissibile* e quel valore comune.

I due bound si incontrano nel *sandwich* $LB \le z(MILP) \le UB$, che percorre tutto il corso: una soluzione ammissibile dà il lato pessimistico, un rilassamento o una soluzione del suo duale danno quello ottimistico.

Tre «gap» da non confondere

La parola *gap* indica tre quantità diverse. Fissiamo che il denominatore sia sempre $|z(MILP)|$ oppure $|UB|$ come indicato, che il numeratore sia in valore assoluto e che, se il denominatore è nullo, il gap relativo non si scriva (si riporta la differenza assoluta).

1. **Gap dell'euristica.** Se $\bar{x}$ è la soluzione costruita dall'euristica, il suo valore $\sum_{j=1}^{n} c_j \bar{x}_j$ è il bound $UB(\bar{x})$ in un minimo (e $LB(\bar{x})$ in un massimo), e il gap è

   $$\frac{\big|UB(\bar{x}) - z(MILP)\big|}{|z(MILP)|}$$

   quando l'ottimo è noto. È il gap riportato nelle tabelle dei bound, perché le istanze del corso si risolvono all'ottimo. Quando l'ottimo *non* è noto lo si calcola rispetto a un bound duale, che di $z(MILP)$ prende il posto: il numero che se ne ottiene è più grande, ed è l'unico che si possa dichiarare.

2. **Divario certificato** fra due bound noti, senza conoscere $z(MILP)$: $(UB - LB)/|UB|$ per un minimo con $UB > 0$. È quello che si può dichiarare quando l'ottimo non è stato dimostrato: garantisce che $z(MILP)$ stia nell'intervallo $[LB, UB]$, non che sia vicino a uno dei due estremi.

3. `MIPGap` **del solver**: la stessa idea, calcolata da Gurobi con i suoi UB (`ObjVal` dell'incumbent) e LB (`ObjBound`), con la sua convenzione $|UB-LB|/|UB|$ e le sue tolleranze. Vale 0 quando l'ottimalità è dimostrata.

Le formule dei punti 2 e 3 richiedono che l'incumbent esista e che il denominatore non sia nullo: sono convenzioni, non identità matematiche, e vanno dichiarate ogni volta che si riporta un numero.

A che cosa servono davvero i due bound

Negli esercizi di questo corso le istanze sono piccole apposta: Gurobi trova $z(MILP)$ in una frazione di secondo, e LB e UB servono a *capire* da dove viene un bound e a verificare il ragionamento sull'euristica e sul duale. Nella pratica, però, un MILP reale può avere migliaia di variabili e il solver può non terminare in tempo utile: lo si interrompe, e resta la migliore soluzione trovata finora e con il miglior bound dimostrato al momento dello stop. Il divario certificato fra i due è tutto quello che si può dire con sicurezza sulla

distanza dall'ottimo, *senza* conoscere $z(MILP)$. Saper costruire in fretta una soluzione ammissibile, e saper costruire a mano un bound ottimistico, sono gli strumenti che restano quando l'ottimo esatto non è raggiungibile, e sono anche quelli che permettono di giudicare i numeri che il solver riporta.

1.4 Tre pattern che tornano ovunque

Molti vincoli MILP con variabili binarie di selezione (indicizzate da un insieme I) appartengono a una di tre famiglie, a seconda del verso:

Nome	Vincolo	Significato
set partitioning	$\sum_{i \in I} x_i = 1$	esattamente un elemento di I è scelto
set packing	$\sum_{i \in I} x_i \leq 1$	al più un elemento di I è scelto
set covering	$\sum_{i \in I} x_i \geq 1$	almeno un elemento di I è scelto

Un vincolo «ogni lavoro a esattamente una macchina» è un *set partitioning*; «al più un lavoro per fascia oraria» è un *set packing*; «ogni zona coperta da almeno un presidio» è un *set covering*, che è poi la traduzione diretta di una clausola OR. Riconoscere il pattern dice subito come si comporta il vincolo: un *covering* ammette che più elementi siano scelti insieme, un *packing* no, un *partitioning* impone l'uno e l'altro insieme.

1.5 Che cosa fa il solver con i due bound

Un MILP con insieme ammissibile limitato si risolve con algoritmi dedicati. Come funzionano — come si esplora lo spazio delle soluzioni, come si separano i tagli, come si sceglie su che cosa ramificare — non è argomento di questo corso: le tecniche risolutive sono materia di un corso a parte. Qui interessa una cosa sola, ed è bene dirla subito: i due bound che questo corso insegna a costruire a mano sono i due lati dello stesso sandwich che il solver cerca di chiudere. Non glieli si passa: il solver se li costruisce da sé, con le sue euristiche e i suoi rilassamenti.

- Il **bound primale** è una soluzione ammissibile, e dà un valore da battere: ogni parte dello spazio che non può fare meglio viene scartata senza esplorarla.
- Il **bound duale** viene dal rilassamento, e dice quanto si può sperare al massimo: più è vicino all'ottimo intero — cioè più la formulazione è stretta — meno lavoro resta da fare.

Il divario fra i due è quello che il solver riporta come `MIPGap`, ed è anche l'unica cosa che si può affermare con sicurezza quando l'ottimo non si raggiunge. Scrivere il modello con una formulazione stretta e saper produrre a mano i due bound è ciò che, di questo corso, serve al momento di risolvere.

1.6 Quello che questo capitolo ha messo in mano

Un modello MIP si scrive sempre con gli stessi cinque oggetti: i **dati**, che sono noti e non si decidono; le **variabili**, una per decisione, con il loro dominio; la **funzione obiettivo**, che dice che cosa si massimizza o si minimizza; i **vincoli**, uno per ogni frase dell'enunciato; e il **dominio** delle variabili, che è un vincolo anche lui, il più facile da dimenticare.

Su un modello scritto si distinguono poi una **soluzione ammissibile**, che rispetta tutti i vincoli, e la **soluzione ottima**, che fra quelle ammissibili ha il valore migliore. Rilassare l'interezza dà il **rilassamento lineare**, che è un problema più facile e il cui ottimo è un **bound**: pessimistico se viene da una soluzione ammissibile, ottimistico se viene da un rilassamento. La distanza fra i due è il **gap**, e il resto del corso serve a costruire i due lati e a stringerli.

Capitolo 2

Rilassamenti, dualità e bound

Script: `python/cap04_bound.py` Pagina online

Questo capitolo insegna a produrre, *a mano*, un numero che sta certamente da una parte dell'ottimo intero. Serve a tre cose: capire quanto vale un modello, capire quanto vale un'euristica, e leggere con giudizio i numeri che un solver riporta quando non ha finito.

2.1 Che cos'è un rilassamento

Definizione operativa

Un **rilassamento** di un problema di minimo con insieme ammissibile X è un problema con lo stesso obiettivo e un insieme ammissibile più grande, $X \subseteq \hat{X}$: l'unica cosa che si rilassa sono i vincoli. Vale allora

$$\min_{x \in \hat{X}} \sum_{j=1}^{n} c_j\, x_j \;\le\; \min_{x \in X} \sum_{j=1}^{n} c_j\, x_j,$$

perché il minimo su un insieme più grande non può essere più alto. In un massimo la disuguaglianza si rovescia.

I tre rilassamenti che il corso usa, in ordine di forza crescente:

Nome	Che cosa si toglie	Nota
$z(LP)$, rilassamento senza i bound	$x \in \{0,1\}$ diventa $x \ge 0$	è quello di cui si scrive il duale a mano: ha meno vincoli e quindi un duale con meno variabili
$z(LP^{+})$, con i bound	$x \in \{0,1\}$ diventa $0 \le x \le 1$	è `relax()` di Gurobi, il primo rilassamento che il solver risolve
$z(LP^{++})$, con i bound e le disuguaglianze valide	come sopra, più disuguaglianze valide	la sezione 2.7 spiega quali

In un problema di minimo $z(LP) \le z(LP^{+}) \le z(LP^{++}) \le z(MILP)$.

> Se il modello contiene un vincolo di assegnamento $\sum_m x_{jm} = 1$ con $x \ge 0$, allora $x_{jm} \le 1$ è già implicato e i due rilassamenti sono *uguali*. Succede ogni volta che un vincolo di uguaglianza costringe già le variabili sotto l'uno, ed è il caso di tutti i modelli di assegnamento.

Quando invece i due differiscono, la differenza va riportata: è un'informazione sul modello.

2.2 La tabella di conversione primale/duale

Da un primale di minimo al suo duale

Sia il primale un minimo, con m vincoli indicizzati da i e n variabili indicizzate da j.

Nel primale (min)	Nel duale (max)
vincolo i di verso $\geq$	variabile $\pi_i \geq 0$
vincolo i di verso $\leq$	variabile $\pi_i \leq 0$
vincolo i di uguaglianza	variabile π_i libera
variabile $x_j \geq 0$	vincolo j di verso $\leq c_j$
variabile x_j libera	vincolo j di uguaglianza $= c_j$

L'obiettivo duale è $\max \sum_i b_i \pi_i$ e il vincolo duale j è $\sum_i a_{ij} \pi_i \lessgtr c_j$ con il verso dato dalla tabella. Se il primale è di **massimo**, tutti i versi della tabella si rovesciano e il duale è di minimo.

Come si legge il duale

Il vincolo duale j dice: «il valore che attribuisco alle risorse consumate dall'attività j non può superare il suo costo». Con questa lettura, ogni ricetta per costruire una soluzione duale ammissibile ha un significato economico, ed è quello che la rende ricordabile: la sezione 2.4 ne dà tre.

2.3 Dualità debole, dualità forte

I due teoremi, e a che cosa servono qui

Siano $(\bar{x}_1, \ldots, \bar{x}_n)$ una soluzione ammissibile per un modello di LP di minimo e $(\bar{\pi}_1, \ldots, \bar{\pi}_m)$ una soluzione ammissibile per il suo duale.

- **Dualità debole**: $\sum_{i=1}^{m} b_i\, \bar{\pi}_i \leq \sum_{j=1}^{n} c_j\, \bar{x}_j$. *Sempre*, senza ipotesi. È questa che ci serve: dà un lower bound da *qualunque* soluzione duale ammissibile, anche costruita a mano e lontana dall'ottimo.
- **Dualità forte**: se l'LP ha ottimo finito, allora $z(D(LP)) = z(LP)$. Serve come *controllo*: l'ottimo del duale scritto a mano deve coincidere con $z(LP)$; se non coincide, il duale è scritto male. Lo script del corso lo verifica con un `assert` in `due_rilassamenti`.

E poi il passaggio all'intero, che è banale ma va detto: siccome ogni soluzione ammissibile del MILP è ammissibile anche per il rilassamento,

$$\sum_{i=1}^{m} b_i\, \bar{\pi}_i \;\leq\; z(LP) \;\leq\; z(MILP).$$

Un bound duale dell'LP è un bound anche per il MILP, dallo stesso lato.

Il duale che si scrive è quello del *rilassamento*, non del MILP. Il MILP non ha un duale lineare, e la dualità forte fra MILP e un qualsiasi programma lineare in generale non vale: il salto $z(MILP) - z(LP)$ è precisamente ciò che manca. Quando in questo corso si dice «il duale», si intende sempre il duale del rilassamento indicato.

2.4 Tre ricette per costruire a mano una soluzione duale

Le ricette

1. **Azzerare e saturare.** Si pongono a zero tutte le variabili duali tranne una famiglia, e si portano quelle al valore più grande (in un duale di massimo) compatibile con i vincoli duali. Esempio: nel problema 7.1 si pone $\bar{\pi} = 0$ e si ottiene $\bar{\mu}_j = \min_m c_{jm}$, cioè «ogni lavoro costa almeno il suo costo minimo».
2. **Euristica costruttiva sui vincoli.** Si scorrono i vincoli primali uno alla volta, si alza la variabile duale corrispondente fino a saturare il primo vincolo duale che si oppone, e si aggiornano i residui. È la ricetta usata qui sotto per la copertura.
3. **Il rapporto migliore.** Con un solo vincolo di capacità in un massimo, la variabile duale $\bar{v} = \max_j p_j/w_j$ è ammissibile, e dà il bound $b\bar{v}$: «tutta la capacità impiegata all'efficienza migliore».

Qualunque ricetta si usi, la soluzione costruita va *verificata* ammissibile per il duale — è l'unica cosa che rende valido il bound — e il suo valore va confrontato con $z(LP)$ per vedere quanto si è lasciato sul tavolo.

2.5 Un problema di minimo, per esteso

Copertura di zone a costo minimo

Un comune ha $n \in \mathbb{Z}_{\geq 1}$ distretti; ciascuno ha una squadra di pronto intervento, e attivare la squadra del distretto $j \in \{1, 2, \ldots, n\}$ costa $c_j \in \mathbb{Q}_{>0}$ migliaia di euro. Ci sono $m \in \mathbb{Z}_{\geq 1}$ zone sensibili: la zona $i \in \{1, 2, \ldots, m\}$ è coperta se almeno una delle squadre dell'insieme S_i dei distretti che la confinano è attiva. Si vogliono coprire tutte le zone a costo minimo.

I dati dell'istanza sono $n = 4$ distretti, di costi

$$c_1 = 4, \qquad c_2 = 3, \qquad c_3 = 5, \qquad c_4 = 3,$$

e $m = 6$ zone, che corrispondono alle sei coppie di distretti confinanti:

zona i	1	2	3	4	5	6
squadre che la coprono, S_i	$\{1,2\}$	$\{2,3\}$	$\{1,3\}$	$\{1,4\}$	$\{2,4\}$	$\{3,4\}$

Si introducono le seguenti n variabili binarie:

$$x_j = \begin{cases} 1 & \text{se la squadra del distretto } j \text{ viene attivata,} \\ 0 & \text{altrimenti,} \end{cases} \qquad \forall j \in \{1, 2, \ldots, n\}.$$

Copertura a costo minimo

$$\min \sum_{j=1}^{n} c_j \, x_j \tag{2.1a}$$

$$\text{soggetto a} \quad \sum_{j \in S_i} x_j \geq 1, \qquad \forall i \in \{1, 2, \ldots, m\}, \tag{2.1b}$$

$$x_j \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{2.1c}$$

Descrizione. L'obiettivo (2.1a) somma il costo delle squadre attivate. Gli m vincoli (2.1b), uno per zona, dicono che almeno una delle due squadre confinanti con quella zona è attiva: è la struttura del *set covering*. I domini (2.1c) sono le n condizioni di binarietà, una per squadra.

Il duale del rilassamento senza i bound. Rilassando (2.1c) a $x_j \geq 0$ e associando a ciascun vincolo (2.1b) una variabile duale $\pi_i \geq 0$ (verso $\geq$ in un minimo, per la tabella di conversione):

Duale del rilassamento LP

$$\max \sum_{i=1}^{m} \pi_i \tag{2.2a}$$

$$\text{soggetto a} \quad \sum_{i \,:\, j \in S_i} \pi_i \leq c_j, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{2.2b}$$

$$\pi_i \geq 0, \qquad \forall i \in \{1, 2, \ldots, m\}. \tag{2.2c}$$

Descrizione. La variabile π_i è il prezzo che si attribuisce alla zona i. L'obiettivo (2.2a) somma i prezzi delle m zone. I vincoli (2.2b) sono le colonne delle x_j, uno per squadra: le zone che la squadra j copre, valutate a quei prezzi, non possono valere più di quanto costa attivarla, cioè c_j. È la lettura economica della sezione precedente: nessuna squadra deve risultare un affare.

Scritti per esteso sui dati dell'istanza, il primale e il duale sono i seguenti.

Copertura sull'istanza — primale

$$
\begin{aligned}
\min\ & 4x_1 + 3x_2 + 5x_3 + 3x_4 \\
& x_1 + x_2 \geq 1 \\
& x_2 + x_3 \geq 1 \\
& x_1 + x_3 \geq 1 \\
& x_1 + x_4 \geq 1 \\
& x_2 + x_4 \geq 1 \\
& x_3 + x_4 \geq 1 \\
& x_1, x_2, x_3, x_4 \in \{0,1\}
\end{aligned}
$$

Copertura sull'istanza — duale del rilassamento LP

$$
\begin{aligned}
\max\ & \pi_1 + \pi_2 + \pi_3 + \pi_4 + \pi_5 + \pi_6 \\
& \pi_1 + \pi_3 + \pi_4 \leq 4 \\
& \pi_1 + \pi_2 + \pi_5 \leq 3 \\
& \pi_2 + \pi_3 + \pi_6 \leq 5 \\
& \pi_4 + \pi_5 + \pi_6 \leq 3 \\
& \pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6 \geq 0
\end{aligned}
$$

Ogni squadra copre tre zone, e infatti ogni vincolo duale ha tre termini: la colonna della variabile x_j nel primale diventa il vincolo j-esimo del duale.

Una soluzione duale a mano (ricetta 2). Si scorrono le zone in ordine; per la zona i si alza π_i del massimo consentito dai residui dei vincoli duali delle squadre in S_i, e si aggiornano i residui.

- **Zona 1** ($S_1 = \{1,2\}$): residui $(4,3,5,3)$, il minimo fra quelli delle squadre 1 e 2 è 3. Si pone $\bar{\pi}_1 = 3$; residui $(1,0,5,3)$.
- **Zona 2** ($\{2,3\}$): il residuo della squadra 2 è 0, quindi $\bar{\pi}_2 = 0$.
- **Zona 3** ($\{1,3\}$): minimo fra 1 e 5, cioè 1. Si pone $\bar{\pi}_3 = 1$; residui $(0,0,4,3)$.
- **Zone 4 e 5** ($\{1,4\}$ e $\{2,4\}$): le squadre 1 e 2 hanno residuo nullo, quindi $\bar{\pi}_4 = \bar{\pi}_5 = 0$.
- **Zona 6** ($\{3,4\}$): minimo fra 4 e 3, cioè 3. Si pone $\bar{\pi}_6 = 3$; residui $(0,0,1,0)$.

Tutti i vincoli duali sono soddisfatti per costruzione, e il valore è

$$LB \;=\; \bar{\pi}_1 + \cdots + \bar{\pi}_6 \;=\; 3+0+1+0+0+3 \;=\; 7.$$

Un upper bound primale. L'euristica costruttiva di copertura sceglie ogni volta la squadra col miglior rapporto fra costo e zone ancora scoperte: prende le squadre 1, 2 e 4, di costo $4+3+3=10$. La soluzione è ammissibile e *intera*, quindi $UB = 10$.

Il sandwich, e cosa dice il solver. $z(LP) = z(LP^+) = 15/2$ (i due rilassamenti coincidono, e l'ottimo del duale li conferma: dualità forte) e $z(MILP) = 10$, ottenuto con le squadre $1, 2, 4$ — la stessa soluzione dell'euristica, che qui è ottima.

UB (euristica costruttiva)	LB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
10	7	15/2	10	0,0%

Da leggere così: il bound a mano vale 7, il rilassamento 7,5, l'ottimo 10. Il divario certificato fra i nostri due bound costruiti a mano è $(10-7)/10 = 30\%$: senza risolvere il MILP sapremmo soltanto che l'ottimo sta fra 7 e 10. L'euristica era già ottima, ma non lo si può sapere dai bound.

2.6 Un problema di massimo: i ruoli si scambiano

Zaino a più risorse

Ci sono $n \in \mathbb{Z}_{\geq 1}$ oggetti e $m \in \mathbb{Z}_{\geq 1}$ risorse: l'oggetto $j \in \{1, 2, \ldots, n\}$ ha valore $p_j \in \mathbb{Q}_{>0}$ e consuma $a_{ij} \in \mathbb{Q}_{\geq 0}$ unità della risorsa $i \in \{1, 2, \ldots, m\}$, di cui sono disponibili $b_i \in \mathbb{Q}_{>0}$ unità. Si scelga il sottoinsieme di valore massimo che sta dentro *tutte* le disponibilità.

I dati dell'istanza sono $n = 5$ lavorazioni, di valore $p = (10, 9, 7, 6, 4)$, e $m = 3$ risorse — acciaio, ore di tornio e ore di collaudo — di disponibilità $b = (10, 9, 8)$. I consumi sono

$$a = \begin{pmatrix} 6 & 4 & 3 & 3 & 2 \\ 1 & 3 & 4 & 2 & 3 \\ 3 & 2 & 2 & 4 & 1 \end{pmatrix},$$

una riga per risorsa e una colonna per lavorazione.

Si introducono le seguenti n variabili binarie:

$$x_j = \begin{cases} 1 & \text{se la lavorazione } j \text{ viene scelta,} \\ 0 & \text{altrimenti,} \end{cases} \qquad \forall j \in \{1, 2, \ldots, n\}.$$

Zaino a più risorse

$$\max \sum_{j=1}^{n} p_j \, x_j \tag{2.3a}$$

$$\text{soggetto a} \sum_{j=1}^{n} a_{ij} \, x_j \leq b_i, \qquad \forall i \in \{1, 2, \ldots, m\}, \tag{2.3b}$$

$$x_j \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{2.3c}$$

Descrizione. L'obiettivo (2.3a) somma il valore delle lavorazioni scelte. I vincoli (2.3b) sono le capacità: uno per risorsa, e ciascuno dice che il consumo complessivo delle lavorazioni scelte non supera la disponibilità. I domini (2.3c) sono le n condizioni di binarietà, una per lavorazione: una lavorazione o si fa tutta, o non si fa.

Il duale del rilassamento senza i bound. Si rilassa il dominio (2.3c) a $x_j \geq 0$. Il primale è di massimo e ha m vincoli, tutti di verso $\leq$: la tabella di conversione dà m variabili duali $v_i \geq 0$, una per risorsa, e un vincolo duale per ciascuna delle n variabili primali.

Duale del rilassamento LP del modello dello zaino

$$\min \sum_{i=1}^{m} b_i\, v_i \tag{2.4a}$$

$$\text{soggetto a} \quad \sum_{i=1}^{m} a_{ij}\, v_i \geq p_j, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{2.4b}$$

$$v_i \geq 0, \qquad \forall i \in \{1, 2, \ldots, m\}. \tag{2.4c}$$

Descrizione. La variabile v_i è il prezzo di una unità della risorsa i. L'obiettivo (2.4a) è il valore attribuito a tutte le disponibilità. Gli n vincoli (2.4b), uno per lavorazione, dicono che le risorse consumate dalla lavorazione j, valutate a quei prezzi, non valgono meno del profitto che quella lavorazione porta. Ogni vincolo duale ha ora m termini, uno per risorsa: la colonna della variabile x_j nel primale ha m righe.

Scritti per esteso sui dati dell'istanza, il primale e il duale sono i seguenti.

Zaino sull'istanza — primale

$$\begin{aligned} \max\ & 10x_1 + 9x_2 + 7x_3 + 6x_4 + 4x_5 \\ \text{soggetto a}\ & 6x_1 + 4x_2 + 3x_3 + 3x_4 + 2x_5 \leq 10 \\ & x_1 + 3x_2 + 4x_3 + 2x_4 + 3x_5 \leq 9 \\ & 3x_1 + 2x_2 + 2x_3 + 4x_4 + x_5 \leq 8 \\ & x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5 \in \{0, 1\} \end{aligned}$$

Zaino sull'istanza — duale del rilassamento LP

$$\begin{aligned} \min\ & 10v_1 + 9v_2 + 8v_3 \\ \text{soggetto a}\ & 6v_1 + v_2 + 3v_3 \geq 10 \\ & 4v_1 + 3v_2 + 2v_3 \geq 9 \\ & 3v_1 + 4v_2 + 2v_3 \geq 7 \\ & 3v_1 + 2v_2 + 4v_3 \geq 6 \\ & 2v_1 + 3v_2 + v_3 \geq 4 \\ & v_1, \quad v_2, \quad v_3 \geq 0 \end{aligned}$$

I due bound.

- **Euristica** (euristica costruttiva per rapporto fra valore e consumo complessivo): i rapporti sono $10/10 = 1$, $9/9 = 1$, $7/9$, $6/9$ e $4/6$; si prendono la lavorazione 1 e poi la 2, e nessun'altra ci sta più perché l'acciaio è finito. Valore $10 + 9 = 19$. In un **massimo** l'euristica dà un **lower** bound: $LB = 19 \leq z(MILP)$.
- **Duale a mano** (ricetta 3, adattata): si paga *una sola* risorsa. Scegliendo la risorsa i e ponendo $\bar{v}_i = \max_j p_j/a_{ij}$ con tutte le altre a zero, ogni vincolo duale è soddisfatto dal solo termine $a_{ij}\bar{v}_i \geq p_j$. Le tre scelte danno $10 \cdot \frac{7}{3} = \frac{70}{3}$ con l'acciaio, $9 \cdot 10 = 90$ col tornio e $8 \cdot \frac{9}{2} = 36$ col collaudo: la più conveniente è l'acciaio. In un **massimo** il duale dà un **upper** bound: $z(MILP) \leq UB = \frac{70}{3} \approx 23{,}33$.

Il sandwich. $z(MILP) = 22$ (lavorazioni 2, 3 e 4, che saturano tutte e tre le risorse), $z(LP) = 159/7 \approx 22{,}71$ e $z(LP^+) = 22$:

$$19 \;\leq\; z(MILP) = 22 \;=\; z(LP^+) \;\leq\; z(LP) = \tfrac{159}{7} \;\leq\; UB = \tfrac{70}{3}.$$

LB (euristica costruttiva)	$z(MILP)$	$z(LP^+)$	$z(LP)$	UB (duale a mano)	gap euristica
19	22	22	159/7	70/3	13,6%

Qui il duale a mano *non* è ottimo per il rilassamento ($70/3 > 159/7$): pagare una sola risorsa è una ricetta valida, ma grossolana, perché ignora che le altre due sono sature anche loro. Il rilassamento con i bound vale 22 e coincide con l'ottimo intero: il vincolo $x_j \leq 1$ morde, perché senza di esso l'LP prende più di una unità della lavorazione migliore.

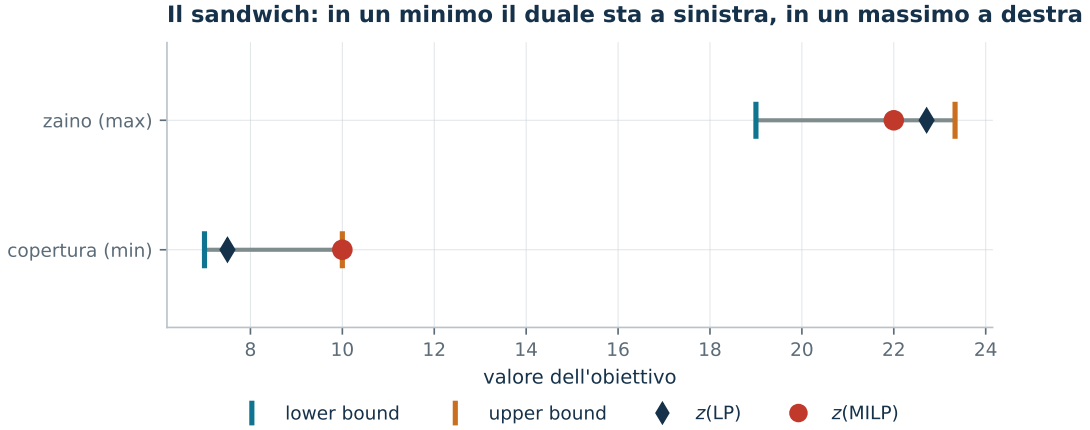


Figura 2.1: I due esempi. In un minimo il bound duale sta a sinistra dell'ottimo e l'euristica a destra; in un massimo i ruoli si scambiano.

Il sandwich scritto una volta per tutte

minimo: $LB(\bar{\pi}) \;\leq\; z(D(LP)) = z(LP) \;\leq\; z(LP^+) \;\leq\; z(MILP) \;\leq\; UB(\bar{x})$

massimo: $LB(\bar{x}) \;\leq\; z(MILP) \;\leq\; z(LP^+) \;\leq\; z(LP) = z(D(LP)) \;\leq\; UB(\bar{\pi})$

dove $(\bar{\pi}_1, \bar{\pi}_2, \ldots, \bar{\pi}_m)$ è una soluzione duale ammissibile del rilassamento, di valore $\sum_{i=1}^{m} b_i\,\bar{\pi}_i$, e $(\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$ una soluzione ammissibile del MILP, di valore $\sum_{j=1}^{n} c_j\,\bar{x}_j$. Le catene si leggono così: il *lato del rilassamento* è ottimistico e contiene tutti i bound duali;

il *lato dell'euristica* è pessimistico e contiene tutte le soluzioni ammissibili. Il nome (LB o UB) dipende dal verso dell'obiettivo, il ruolo no.

2.7 Disuguaglianze valide e vincoli che preservano l'ottimalità

Due nozioni diverse

- Una **disuguaglianza valida** per un MILP è soddisfatta da *tutte* le soluzioni ammissibili intere. Aggiungerla non cambia $z(MILP)$; può ridurre $z(LP^+)$, e allora si chiama **taglio**.
- Un **vincolo che preserva l'ottimalità** taglia alcune soluzioni ammissibili, ma non tutte quelle ottime: dopo l'aggiunta esiste ancora un ottimo, e $z(MILP)$ non cambia. Non è una disuguaglianza valida, e va dichiarato come tale. Esempio: sia z una variabile che l'obiettivo minimizza e che compare solo in vincoli $z \geq t_j x_j$. Imporre in più $z \leq My$, con y una binaria di attivazione, taglia soluzioni ammissibili — quelle con $y = 0$ e z positiva — ma non quelle ottime, perché all'ottimo z è già tirata in basso dall'obiettivo.

La differenza conta: la prima si può aggiungere sempre; la seconda soltanto se si è dimostrato che almeno un ottimo sopravvive, e mai se poi si vuole enumerare *tutte* le soluzioni ottime.

Il taglio di copertura per lo zaino

Un insieme S di oggetti è una **copertura** se $\sum_{j \in S} w_j > b$: non ci stanno tutti insieme. Allora

$$\sum_{j \in S} x_j \;\leq\; |S| - 1$$

è una disuguaglianza valida. La copertura è **minimale** se nessun suo sottoinsieme proprio è già una copertura; i tagli delle coperture minimali sono i più forti.

Sull'istanza dello zaino ($w = (5, 4, 3, 3)$, $b = 9$) le coperture minimali sono le quattro terne $\{1,2,3\}$, $\{1,2,4\}$, $\{1,3,4\}$, $\{2,3,4\}$, ciascuna di peso 12, 12, 11, 10: tutte > 9, e nessuna coppia lo è. I quattro tagli sono $\sum_{j \in S} x_j \leq 2$.

La soluzione ottima del rilassamento $z(LP^+) = 71/4$ è $\tilde{x} = (1,\ 1/4,\ 1,\ 0)$. Su di essa:

Copertura S	$\sum_{j \in S} \tilde{x}_j$	$\lvert S\rvert - 1$	
$\{1,2,3\}$	$9/4$	2	**violato**: il taglio serve
$\{1,2,4\}$	$5/4$	2	soddisfatto
$\{1,3,4\}$	2	2	soddisfatto (all'uguaglianza)
$\{2,3,4\}$	$5/4$	2	soddisfatto

Aggiungendo tutti e quattro i tagli, $z(LP^+)$ scende da $71/4 = 17{,}75$ a $69/4 = 17{,}25$, e $z(MILP)$ resta 17: il rilassamento si è avvicinato all'ottimo intero senza cambiarlo. È il meccanismo con cui un solver moderno chiude i gap.

2.8 Formulazioni più forti: che cosa vuol dire, e come si dimostra

Il criterio

Due formulazioni A e B dello stesso problema si confrontano in due passi.

1. **Stesso insieme intero**: $X_A \cap \mathbb{Z}^n = X_B \cap \mathbb{Z}^n$. Senza questo passaggio non si sta confrontando nulla. Si dimostra nei due versi, usando la binarietà o l'interezza dove serve.

2. **Rilassamento**: B è *più forte* di A se $X_B \subseteq X_A$ come poliedri, e allora $z_B(LP^+)$ è almeno buono quanto $z_A(LP^+)$. Se l'inclusione è stretta *e* si esibisce un punto di $X_A \setminus X_B$ ottimo per il rilassamento di A, il miglioramento è dimostrato su quell'istanza.

Il caso di riferimento è l'**attivazione**, il legame fra «il cliente i è servito dalla sede j» (x_{ij}) e «la sede j è aperta» (y_j): si può scrivere disaggregata, $x_{ij} \leq y_j$ per ogni coppia, oppure aggregata, $\sum_i x_{ij} \leq n y_j$ per ogni sede. La prima implica la seconda, quindi il disaggregato è più forte; su tre clienti e due sedi il rilassamento passa da 38/3 a 15, che è già l'ottimo intero.

Una formulazione più forte ha meno nodi ma righe in più, e ogni nodo costa di più. Il bilancio dipende dall'istanza. Quello che si può *dimostrare* è la forza del rilassamento; la velocità si *misura*.

2.9 Quello che dice il solver

I quattro attributi da leggere, e in quale ordine

1. `Status`: 2 è `OPTIMAL`, 3 `INFEASIBLE`, 9 `TIME_LIMIT`. Va letto *prima* di ogni altra cosa.

2. `SolCount`: quante soluzioni intere sono state trovate. Se è 0, `ObjVal` non esiste e leggerlo è un errore.

3. `ObjVal` e `ObjBound`: la migliore soluzione trovata e il miglior bound dimostrato. In un minimo, `ObjBound` $\leq z(MILP) \leq$ `ObjVal`.

4. `MIPGap` e `NodeCount`: il divario certificato fra i due e quanti nodi sono stati esplorati.

Sull'istanza di copertura, il rilassamento del modello *come lo abbiamo scritto* vale 15/2 e l'ottimo intero 10. Eppure Gurobi riporta `ObjBound` = 10 e `NodeCount` = 0: ha chiuso il gap nella radice, con presolve, tagli propri ed euristiche, senza mai ramificare. Spegnendoli (`Presolve = Cuts = Heuristics = 0`) lo stesso modello dà lo stesso ottimo 10 ma con 5 nodi. Due conseguenze:

- «quanto è difficile un modello» non è una proprietà del solo modello: dipende anche da che cosa il solver mette in campo;

- il rilassamento di cui parliamo nei bound a mano è quello del modello scritto, e va ottenuto con `relax()`: leggere `ObjBound` a fine risoluzione dà l'ottimo, non il rilassamento.

2.10 I duali dell'LP non sono i prezzi marginali del MILP

L'affermazione, e il controesempio

In un LP, il duale $\tilde{\pi}_i$ di un vincolo di risorsa è il suo *prezzo marginale*: aumentando b_i di una piccola quantità, l'ottimo varia di $\tilde{\pi}_i$ per unità (finché la base non cambia). **Per un MILP questo è falso**, e non per approssimazione.
Sullo zaino, al variare della capacità b:

b	$z(MILP)$	$z(LP^+)$	duale dell'LP	variazione vera di $z(MILP)$
8	16	16	2	—
9	17	71/4	7/4	+1
10	17	39/2	7/4	**0**
11	20	85/4	7/4	+3
12	23	23	7/4	+3

Il duale dell'LP è il rapporto p_j/w_j dell'oggetto «critico»: vale 2 quando la capacità si esaurisce sull'oggetto 1 e 7/4 quando avanza spazio per il 2. La variazione vera dell'ottimo intero è a scatti: passando da $b = 9$ a $b = 10$ non cambia *affatto* (nessun sottoinsieme sfrutta l'unità in più), mentre il duale promette 7/4; passando da 10 a 11 salta di 3.

Del duale dell'LP resta vero l'unico uso che questo corso ne fa: è un **bound**. $\tilde{\pi}_i$ dice quanto vale la risorsa *nel rilassamento*, e la somma $b'\tilde{u}$ è un bound valido per il MILP. Come indicazione gestionale («conviene comprare un'unità in più?») va invece verificata risolvendo di nuovo il MILP: la differenza $z(MILP)(b_i+1) - z(MILP)(b_i)$ è l'unica risposta corretta, e non c'è una formula chiusa che la dia.

2.11 Il protocollo dei bound

Davanti a un modello qualunque, il percorso per stringere il sandwich è sempre lo stesso, e conviene farlo nell'ordine:

1. una soluzione **ammissibile e intera**, costruita a mano, verificata su tutti i vincoli *e* sull'interezza — è il controllo di interezza a rendere quel valore un bound del MILP e non del solo rilassamento;
2. il **duale del rilassamento senza i bound**, scritto in forma generale e poi sull'istanza, con i versi presi dalla tabella di conversione;
3. una **soluzione duale ammissibile costruita a mano**, con la ricetta dichiarata, verificata ammissibile e valutata;
4. i due **rilassamenti** $z(LP)$ e $z(LP^+)$, con l'annotazione se coincidono e perché;

5. l'**ottimo** $z(MILP)$ e la tabella $UB \cdot LB \cdot z(LP) \cdot z(LP^{+}) \cdot z(MILP)$ con il gap, definito come nel capitolo 1;

6. le **considerazioni aggiuntive**: disuguaglianze valide con il loro conteggio, vincoli che preservano l'ottimalità dichiarati come tali, confronto dei rilassamenti fra formulazioni alternative.

Il passo 1 e il passo 3 sono i due lati del sandwich, e sono gli unici che si possono costruire senza un solver. Il passo 4 e il passo 5 servono a misurare quanto i primi due erano buoni.

Capitolo 3

Dal modello a Python/Gurobi

Script: `python/cap06_gurobi.py` Pagina online

Il corso usa **un solo solver**, Gurobi da Python. Questo capitolo mostra come si scrive un modello — una famiglia di vincoli per blocco, con i nomi del modello matematico — e soprattutto come si *leggono* i risultati, incluso il caso in cui il solver non ha finito.

3.1 Le istruzioni di base di gurobipy

Un modello si scrive in gurobipy con poche istruzioni, sempre le stesse, e da qui in poi accanto a ogni modello comparira' la riga di codice che lo scrive. Conviene vederle una volta su un modello piccolo e completo: lo **zaino binario**, dove di n oggetti di valore p_j e peso w_j si sceglie un sottoinsieme di peso al più c che valga il più possibile.

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} p_j \, x_j \\ \text{soggetto a} \quad & \sum_{j=1}^{n} w_j \, x_j \le c, \\ & x_j \in \{0,1\}, \qquad \forall j \in \{1,2,\dots,n\}. \end{aligned}$$

```
import gurobipy as gp                          # il modulo, sempre abbreviato gp
from gurobipy import GRB                       # le costanti: GRB.BINARY, GRB.MAXIMIZE...

p = [10, 7, 6, 4]                              # valori
w = [5, 4, 3, 3]                               # pesi
c = 9                                          # capacita'
n = len(p)

m = gp.Model("zaino")                                          # un modello vuoto
x = m.addVars(n, vtype=GRB.BINARY, name="x")                   # n variabili binarie
m.setObjective(gp.quicksum(p[j] * x[j] for j in range(n)), GRB.MAXIMIZE)
```

```
m.addConstr(gp.quicksum(w[j] * x[j] for j in range(n)) <= c, name="
    capacita")
m.optimize()
```

Le istruzioni, una per riga del modello:

- `Model` crea il modello; tutto il resto si appende a lui.
- `addVars` aggiunge un'intera famiglia indicizzata: `addVars(n)` dà $x_0, \ldots, x_{n-1}$, mentre `addVars(n, k)` dà la matrice x_{ij}, e si indicizzano con `x[j]` e `x[i, j]`. Per una variabile sola c'è `addVar`. Gli indici di Python partono da zero, nei modelli scritti partono da uno: è la sola differenza da tenere a mente.
- `vtype` dichiara il dominio: `GRB.BINARY` per $x \in \{0, 1\}$, `GRB.INTEGER` per $x \in \mathbb{Z}_{\geq 0}$, e senza `vtype` la variabile è continua e non negativa. Il dominio è un vincolo: dichiararlo male è dichiarare un altro modello.
- `quicksum` costruisce una somma $\sum_j c_j x_j$; fa lo stesso di `sum` ma senza costruire espressioni intermedie, ed è quella che si usa sempre.
- `setObjective` prende l'espressione e il verso, `GRB.MINIMIZE` o `GRB.MAXIMIZE`.
- `addConstr` aggiunge *un* vincolo, `addConstrs` una famiglia, e vuole un generatore fra parentesi tonde: una riga di codice per una famiglia di vincoli, com'è una riga sola nel modello scritto.
- `optimize` risolve.

Il `name` non serve al solver: serve a chi legge il modello esportato o un messaggio di errore, e vale la pena darlo sempre uguale al nome della famiglia di vincoli nel modello scritto.

Con le stesse istruzioni si scrivono gli altri problemi classici che tornano in queste pagine. Il **bin packing** — n oggetti di dimensione w_j in contenitori di capacità c, usandone il meno possibile — vuole due famiglie di binarie, x_{ji} per «l'oggetto j va nel contenitore i» e y_i per «il contenitore i si usa», e due famiglie di vincoli:

```
x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
y = m.addVars(k, vtype=GRB.BINARY, name="y")
m.setObjective(y.sum(), GRB.MINIMIZE)
m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna")
m.addConstrs((gp.quicksum(w[j] * x[j, i] for j in range(n)) <= c * y[i]
              for i in range(k)), name="capacita")
```

Lo **scheduling su macchine identiche** ($P||z$) cambia solo l'obiettivo e un vincolo: una continua T per il makespan, $\min T$, e $\sum_j t_j x_{ji} \leq T$ per ogni macchina. Il **commesso viaggiatore** ha una binaria per arco, x_{ij}, e due famiglie di uguaglianze — da ogni città si parte una volta e in ogni città si arriva una volta — più i vincoli che impediscono i sottogiri, che sono la parte interessante e non servono qui.

3.2 Le quattro classi di variabili

Che cosa	vtype	Dominio	Uso tipico
decisione sì/no	`GRB.BINARY`	$\{0, 1\}$	selezione, attivazione, assegnamento
conteggio	`GRB.INTEGER`	$\mathbb{Z}$ fra `lb` e `ub`	scatole, turni, operai
quantità misurabile	(default)	$[\texttt{lb}, \texttt{ub}] \subseteq \mathbb{R}$	tempo, denaro, flusso
variabile libera	(default) con `lb=-GRB.INFINITY`	$\mathbb{R}$	duali di uguaglianze

`GRB.BINARY` implica già `lb` $= 0$ e `ub` $= 1$: non serve ripeterli. Una variabile continua ha `lb = 0` per default: una variabile che deve poter essere negativa — tipicamente un duale di uguaglianza, o una deviazione con segno — va dichiarata con `lb=-GRB.INFINITY`, altrimenti il modello è silenziosamente sbagliato.

3.3 Il modello, una famiglia per blocco

```
def modello(t, c, a):
    """Un assegnamento: una addConstrs per famiglia, con il nome dell'
       etichetta."""
    m = gp.Model("assegnamento");  m.Params.OutputFlag = 0
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")            # dati ->
        variabili
    m.setObjective(gp.quicksum(c[j][h] * x[j, h] for j in range(n)
                               for h in range(k)), GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna"
       )
    m.addConstrs((gp.quicksum(t[j][h] * x[j, h] for j in range(n)) <= a
       [h]
                  for h in range(k)), name="disponibilita")
    return m, x
```

Le tre regole di scrittura del corso

1. **Una `addConstrs` per famiglia**, nell'ordine del modello matematico, e il `name` è l'etichetta della famiglia (`"assegna"`, `"disponibilita"`). I nomi generati sono `assegna[0]`, `assegna[1]`, …: si ritrovano nei duali e nei messaggi del solver.
2. **Le variabili si dichiarano tutte insieme**, con `addVars` e gli indici del modello. `x.sum(j, "*")` è la somma su tutti gli indici della seconda dimensione: è la scrittura dell'*indice muto* del modello.
3. **I dati arrivano come argomenti**, non come variabili globali: la stessa funzione serve per l'istanza base e per tutte le varianti delle domande di modellazione.

Il modello scritto ha 9 variabili, 6 vincoli e 18 coefficienti non nulli. Per controllare che sia quello che si è scritto sulla carta, `m.write("modello.lp")` produce il formato LP leggibile:

```
Minimize
  5 x[0,0] + 10 x[0,1] + 2 x[0,2] + 5 x[1,0] + 4 x[1,1] + 6 x[1,2]
   + 5 x[2,0] + 4 x[2,1] + 6 x[2,2]
Subject To
 assegna[0]: x[0,0] + x[0,1] + x[0,2] = 1
 assegna[1]: x[1,0] + x[1,1] + x[1,2] = 1
 assegna[2]: x[2,0] + x[2,1] + x[2,2] = 1
 disponibilita[0]: 2 x[0,0] + 3 x[1,0] + 4 x[2,0] <= 5
 disponibilita[1]: x[0,1] + 4 x[1,1] + 5 x[2,1] <= 6
 disponibilita[2]: 3 x[0,2] + 2 x[1,2] + 3 x[2,2] <= 7
Bounds
Binaries
 x[0,0] x[0,1] x[0,2] x[1,0] x[1,1] x[1,2] x[2,0] x[2,1] x[2,2]
End
```

È il modo più rapido per accorgersi di un coefficiente sbagliato: il tabulare dell'istanza nella dispensa e questo output devono coincidere riga per riga.

3.4 Leggere i risultati

L'ordine di lettura, che non si cambia

1. `Status`. Sempre per primo.
2. `SolCount`: se vale 0, *non esiste* una soluzione e `ObjVal` non si può leggere.
3. `ObjVal` e `ObjBound`: la migliore soluzione trovata e il miglior bound dimostrato.
4. `MIPGap`, `NodeCount`, `Runtime`: quanto resta incerto, e quanto è costato.

`Status`	valore	`SolCount`	Che cosa si può dire
`OPTIMAL`	2	≥ 1	$z(MILP) =$ `ObjVal`, dimostrato
`INFEASIBLE`	3	0	il modello non ha soluzioni ammissibili
`UNBOUNDED`	5	0	manca un vincolo, o un bound su una variabile
`TIME_LIMIT`	9	0	niente: né una soluzione né, in generale, un bound utile
`TIME_LIMIT`	9	≥ 1	`ObjBound` $\leq z(MILP) \leq$ `ObjVal`
`SOLUTION_LIMIT`	10	≥ 1	come sopra

I quattro casi sull'istanza di assegnamento

- **Risoluzione normale.** `Status` $= 2$, `SolCount` $= 2$, `ObjVal` $=$ `ObjBound` $= 11$, `MIPGap` $= 0$, `NodeCount` $= 0$: Gurobi ha chiuso tutto nella radice.
- **Inammissibile.** Con disponibilità $a = (1, 1, 1)$ nessun assegnamento è possibile: `Status` $= 3$, `SolCount` $= 0$.
- **Fermato subito.** Con `TimeLimit = 0`: `Status` $= 9$, `SolCount` $= 0$ e `ObjBound` $= -\infty$. Non c'è nulla da riportare: né una soluzione né un bound.

- **Fermato alla prima soluzione.** Con `SolutionLimit = 1`: `Status` $= 10$, `SolCount` $= 1$, `ObjVal` $= 12$, `ObjBound` $= 10$, `MIPGap` $= 0{,}1667$. Questo è il caso in cui si riporta un *intervallo*: l'ottimo sta fra 10 e 12, e il 16,7% ne misura l'ampiezza. Dire «l'ottimo è 12» sarebbe falso.

Non si scrive «l'ottimo è `ObjVal`» se `Status` non è `OPTIMAL`. Non si scrive un gap se `SolCount` è 0. Non si confronta il `Runtime` di due modelli risolti con impostazioni diverse. E non si legge `ObjBound` a fine risoluzione pensando di leggere il rilassamento della radice: per quello serve `relax()`, come nella sezione 2.9.

3.5 Le tolleranze

Parametro	Default	Significato
`IntFeasTol`	10^{-5}	quanto una variabile intera può distare da un intero
`FeasibilityTol`	10^{-6}	violazione ammessa su un vincolo lineare
`OptimalityTol`	10^{-6}	tolleranza sui costi ridotti
`MIPGap`	10^{-4}	gap relativo sotto il quale il solver si ferma

Una binaria può tornare come 0,9999999997. Nel testo si scrive 1: i valori si arrotondano *quando si riportano*, e i confronti si fanno sempre con una tolleranza — nel corso, 10^{-6}, la costante `TOL` di `python/mip.py`. Scrivere `if x.X == 1` è un errore; si scrive `if x.X > 0.5`. Il default `MIPGap` $= 10^{-4}$ significa anche che il solver può fermarsi *prima* dell'ottimo esatto, dichiarando `OPTIMAL`: su istanze con valori grandi vale la pena abbassarlo.

3.6 Il rilassamento con `relax()`

```
m.update()              # relax() copia il modello: le modifiche pendenti
    vanno applicate prima
r = m.relax()           # le binarie diventano 0 <= x <= 1, le intere x
   >= lb
r.Params.OutputFlag = 0
r.optimize()
zlp = r.ObjVal
duali = {c.ConstrName: c.Pi for c in r.getConstrs()}
```

Su quell'istanza, $z(LP^+) = z(LP) = 53/5$ — i due rilassamenti coincidono, perché i vincoli di assegnamento implicano già $x_{jm} \le 1$ — e i duali non nulli sono

$$\tilde{\mu} = (2,\ 4{,}8,\ 5), \qquad \tilde{\pi}_2 = -0{,}2,$$

cioè le tre variabili duali dei vincoli `assegna` e quella del vincolo `disponibilita[1]`: la macchina 2 è la sola risorsa stretta.

Il rilassamento senza i bound, quando serve

La funzione `rilassamento(m, rafforzato=False)` di `python/mip.py` toglie anche il limite superiore delle binarie: è il rilassamento di cui la dispensa scrive il duale a mano. Le due versioni si riportano entrambe, con l'annotazione se coincidono e perché.

3.7 Tre problemi che il corso riusa

Fin qui il modello di esempio è sempre stato lo zaino. I tre problemi di questa sezione ritornano nel capitolo 4, dove si costruiscono a mano le soluzioni di next-fit, first-fit, best-fit, LPT e vicino più vicino: qui si scrivono i loro modelli, così quelle euristiche hanno un ottimo con cui confrontarsi.

Bin packing: quanti contenitori bastano

Script: `python/cap06_bpp.py`

Bin packing

Ci sono $n \in \mathbb{Z}_{\geq 1}$ oggetti: l'oggetto $j \in \{1, 2, \ldots, n\}$ pesa $w_j \in \mathbb{Q}_{>0}$. I contenitori sono tutti uguali, di capacità $c \in \mathbb{Q}_{>0}$, e se ne può usare al più uno per oggetto. Si usi il minimo numero di contenitori.

Il modello

Servono due famiglie di variabili binarie: $x_{jb} = 1$ se l'oggetto j va nel contenitore b, e $y_b = 1$ se il contenitore b viene usato.

Bin packing

$$\min \quad \sum_{b=1}^{k} y_b \tag{3.1a}$$

$$\text{soggetto a} \quad \sum_{b=1}^{k} x_{jb} = 1, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{3.1b}$$

$$\sum_{j=1}^{n} w_j \, x_{jb} - c \, y_b \leq 0, \qquad \forall b \in \{1, 2, \ldots, k\}, \tag{3.1c}$$

$$x_{jb} \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}, \ \forall b \in \{1, 2, \ldots, k\}, \tag{3.1d}$$

$$y_b \in \{0, 1\}, \qquad \forall b \in \{1, 2, \ldots, k\}. \tag{3.1e}$$

Descrizione. L'obiettivo (3.1a) conta i contenitori accesi. I vincoli (3.1b) dicono che ogni oggetto finisce in esattamente un contenitore. I vincoli (3.1c) sono la capacità scritta come *attivazione*: finché $y_b = 0$ il contenitore b non può ricevere niente, e appena $y_b = 1$ accoglie fino a c. I domini (3.1d) e (3.1e) sono le due famiglie di binarie, una per coppia oggetto-contenitore e una per contenitore.

La costruzione in gurobipy

```
def modello_bpp(w, c, k):
    n = len(w)
    m = nuovo_modello("bin_packing")
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
    y = m.addVars(k, vtype=GRB.BINARY, name="y")
    m.setObjective(y.sum(), GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in R(n)), name="oggetto")
    m.addConstrs((gp.quicksum(w[j] * x[j, b] for j in R(n)) <= c * y[b]
                  for b in R(k)), name="capacita")
    return m, x, y
```

L'istanza

Sull'istanza di quattro oggetti di peso $w = (5, 4, 3, 3)$ e capacità $c = 7$:

Bin packing sull'istanza

$$
\begin{array}{lrrrrrrrrrrrrrl}
\min & & & & & & & & & & & y_1 & +y_2 & +y_3 & \\
\text{soggetto a} & x_{11} & +x_{12} & +x_{13} & & & & & & & & & & & = 1 \\
& & & & x_{21} & +x_{22} & +x_{23} & & & & & & & & = 1 \\
& & & & & & & x_{31} & +x_{32} & +x_{33} & & & & & = 1 \\
& & & & & & & & & & x_{41} & +x_{42} & +x_{43} & & = 1 \\
& 5x_{11} & & & +4x_{21} & & & +3x_{31} & & & +3x_{41} & & & -7y_1 & \le 0 \\
& & 5x_{12} & & & +4x_{22} & & & +3x_{32} & & & +3x_{42} & & -7y_2 & \le 0 \\
& & & 5x_{13} & & & +4x_{23} & & & +3x_{33} & & & +3x_{43} & -7y_3 & \le 0 \\
& x_{11}, & x_{12}, & x_{13}, & x_{21}, & x_{22}, & x_{23}, & x_{31}, & x_{32}, & x_{33}, & x_{41}, & x_{42}, & x_{43} & & \in \{0,1\} \\
& & & & & & & & & & & & & y_1, \; y_2, \; y_3 & \in \{0,1\}
\end{array}
$$

Il peso totale è 15, quindi nessuna soluzione può usare meno di $\lceil 15/7 \rceil = 3$ contenitori; l'ottimo ne usa esattamente 3, e il conteggio è quindi stretto. Nel modello scritto sopra i contenitori possibili sono tre, il numero che serve davvero: con sei contenitori il modello avrebbe 42 colonne e le ultime tre resterebbero vuote in ogni soluzione ottima.

> Rilassando y_b a $y_b \ge 0$ il modello compra frazioni di contenitore, e l'ottimo dell'LP scende a $\sum_j w_j / c = 15/7 \approx 2{,}14$: il rilassamento non sa che un contenitore si apre tutto intero. È il motivo per cui su questo problema il bound duale è poco utile e le euristiche contano di più.

Makespan: il carico della macchina piu' carica

Script: `python/cap06_cmax.py`

Makespan su macchine identiche

Ci sono $n \in \mathbb{Z}_{\geq 1}$ lavori, di durata $d_j \in \mathbb{Q}_{>0}$, e $k \in \mathbb{Z}_{\geq 1}$ macchine identiche. Ogni lavoro va su una macchina sola e non si interrompe. Si minimizzi il carico della macchina più carica.

Il modello

Con $x_{jm} = 1$ se il lavoro j va sulla macchina m, e $z \geq 0$ il carico della macchina più carica:

Makespan su macchine identiche

$$\min \quad z \tag{3.2a}$$

$$\text{soggetto a} \quad \sum_{m=1}^{k} x_{jm} = 1, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{3.2b}$$

$$\sum_{j=1}^{n} d_j \, x_{jm} - z \leq 0, \qquad \forall m \in \{1, 2, \ldots, k\}, \tag{3.2c}$$

$$x_{jm} \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}, \ \forall m \in \{1, 2, \ldots, k\}, \tag{3.2d}$$

$$z \geq 0. \tag{3.2e}$$

Descrizione. L'obiettivo (3.2a) è la sola variabile z: nessun dato vi compare. Sono le k righe di carico (3.2c) a darle significato, dicendo che nessuna macchina lavora più a lungo di z; il minimo la schiaccia allora sul carico della macchina più carica. È la tecnica *min-max* del capitolo 6. I vincoli (3.2b) assegnano ogni lavoro a una macchina sola, e i domini (3.2d) e (3.2e) dichiarano binarie le assegnazioni e non negativo il carico.

La costruzione in gurobipy

```
def modello_cmax(d, k):
    n = len(d)
    m = nuovo_modello("makespan")
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
    z = m.addVar(name="z")
    m.setObjective(z, GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in R(n)), name="lavoro")
    m.addConstrs((gp.quicksum(d[j] * x[j, mm] for j in R(n)) <= z
                  for mm in R(k)), name="carico")
    return m, x, z
```

L'istanza

Sull'istanza di quattro lavori di durata $d = (3, 4, 5, 6)$ su $k = 2$ macchine — la stessa su cui il capitolo delle euristiche fa correre LPT:

Makespan sull'istanza

$$
\begin{aligned}
\min \quad & & & & & & & & & & z & \\
\text{soggetto a} \quad & x_{11} & +x_{12} & & & & & & & & & = 1 \\
& & & x_{21} & +x_{22} & & & & & & & = 1 \\
& & & & & x_{31} & +x_{32} & & & & & = 1 \\
& & & & & & & x_{41} & +x_{42} & & & = 1 \\
& 3x_{11} & & +4x_{21} & & +5x_{31} & & +6x_{41} & & & -z & \le 0 \\
& & 3x_{12} & & +4x_{22} & & +5x_{32} & & +6x_{42} & & -z & \le 0 \\
& x_{11}, & x_{12}, & x_{21}, & x_{22}, & x_{31}, & x_{32}, & x_{41}, & x_{42} & & & \in \{0,1\} \\
& & & & & & & & & & z & \ge 0
\end{aligned}
$$

Il carico totale è 18 e le macchine sono due: nessuna soluzione può scendere sotto $18/2 = 9$, e l'ottimo vale esattamente 9 — i lavori si dividono in $6+3$ e $5+4$. Qui il conteggio chiude il problema da solo.

Commesso viaggiatore: la formulazione MTZ

Script: `python/cap06_tsp.py`

Commesso viaggiatore

Ci sono $n \in \mathbb{Z}_{\ge 2}$ città e una distanza $d_{ij} \in \mathbb{Q}_{>0}$ fra ogni coppia. Si trovi il giro di lunghezza minima che tocca ogni città esattamente una volta e torna al punto di partenza.

Il modello

Con $x_{ij} = 1$ se il giro va da i a j, le due famiglie «si esce una volta» e «si entra una volta» non bastano: ammettono anche soluzioni fatte di *sottocicli* separati. La formulazione di Miller–Tucker–Zemlin aggiunge una variabile u_i per ogni città diversa dalla prima, che ne registra la posizione lungo il giro.

TSP, formulazione MTZ

$$
\min \quad \sum_{i=1}^{n} \sum_{j \ne i} d_{ij}\, x_{ij} \tag{3.3a}
$$

$$
\text{soggetto a} \quad \sum_{j \ne i} x_{ij} = 1, \qquad \forall i \in \{1, 2, \dots, n\}, \tag{3.3b}
$$

$$
\sum_{i \ne j} x_{ij} = 1, \qquad \forall j \in \{1, 2, \dots, n\}, \tag{3.3c}
$$

$$
u_i - u_j + n\, x_{ij} \le n - 1, \qquad \forall i, j \in \{2, 3, \dots, n\},\ i \ne j, \tag{3.3d}
$$

$$
x_{ij} \in \{0, 1\}, \qquad \forall i, j \in \{1, 2, \dots, n\},\ i \ne j, \tag{3.3e}
$$

$$
u_i \in [1,\, n-1], \qquad \forall i \in \{2, 3, \dots, n\}. \tag{3.3f}
$$

Descrizione. L'obiettivo (3.3a) somma le distanze degli archi percorsi. I vincoli (3.3b) e (3.3c) dicono che da ogni città si esce una volta e in ogni città si entra una volta. I vincoli (3.3d) sono il cuore della formulazione. Se $x_{ij} = 0$ la riga diventa $u_i - u_j \leq n - 1$, sempre vera perché le u stanno fra 1 e $n-1$: non vieta niente. Se invece $x_{ij} = 1$ diventa $u_j \geq u_i + 1$, cioè «se vado da i a j, la posizione di j è la successiva». Un sottociclo che non tocca la città 1 richiederebbe una catena di posizioni sempre crescenti che si richiude su se stessa, e questo è impossibile; la città 1 non ha la sua u proprio perché è il punto in cui il giro si chiude.

La costruzione in gurobipy

```
def modello_tsp(D):
    n = len(D)
    m = nuovo_modello("tsp")
    x = m.addVars(((i, j) for i in R(n) for j in R(n) if i != j),
                  vtype=GRB.BINARY, name="x")
    u = m.addVars(R(1, n), lb=1, ub=n - 1, name="u")
    m.setObjective(gp.quicksum(D[i][j] * x[i, j] for i, j in x), GRB.
        MINIMIZE)
    m.addConstrs((gp.quicksum(x[i, j] for j in R(n) if j != i) == 1
                  for i in R(n)), name="esce")
    m.addConstrs((gp.quicksum(x[i, j] for i in R(n) if i != j) == 1
                  for j in R(n)), name="entra")
    m.addConstrs((u[i] - u[j] + n * x[i, j] <= n - 1
                  for i in R(1, n) for j in R(1, n) if i != j), name="
                     mtz")
    return m, x, u
```

L'istanza

Sull'istanza di quattro città del capitolo 4 il giro ottimo è $1 \to 2 \to 4 \to 3 \to 1$ e misura 22. Il modello dell'istanza ha quindici colonne — dodici archi e tre posizioni — e quattordici righe:

TSP sull'istanza

$$
\begin{aligned}
&\min 4x_{12}+5x_{13}+9x_{14}+4x_{21}+9x_{23}+9x_{24}+5x_{31}+9x_{32}+4x_{34}+9x_{41}+9x_{42}+4x_{43} \\
&\text{soggetto a } x_{12}+x_{13}+x_{14} = 1 \\
&x_{21}+x_{23}+x_{24} = 1 \\
&x_{31}+x_{32}+x_{34} = 1 \\
&x_{41}+x_{42}+x_{43} = 1 \\
&x_{21}+x_{31}+x_{41} = 1 \\
&x_{12}+x_{32}+x_{42} = 1 \\
&x_{13}+x_{23}+x_{43} = 1 \\
&x_{14}+x_{24}+x_{34} = 1 \\
&4x_{23}+u_2-u_3 \le 3 \\
&4x_{24}+u_2-u_4 \le 3 \\
&4x_{32}-u_2+u_3 \le 3 \\
&4x_{34}+u_3-u_4 \le 3 \\
&4x_{42}-u_2+u_4 \le 3 \\
&4x_{43}-u_3+u_4 \le 3 \\
&x_{12},\ x_{13},\ x_{14},\ x_{21},\ x_{23},\ x_{24},\ x_{31},\ x_{32},\ x_{34},\ x_{41},\ x_{42},\ x_{43} \in \{0,1\} \\
&u_2,\ u_3,\ u_4 \ge 1
\end{aligned}
$$

I vincoli MTZ sono $O(n^2)$ e si scrivono in tre righe di `gurobipy`, ma il loro rilassamento lineare è debole: le u continue assorbono quasi tutto e l'LP si avvicina poco all'ottimo intero. Le formulazioni che eliminano i sottocicli con i tagli di connessione danno bound molto migliori, al prezzo di un numero esponenziale di vincoli da generare a mano a mano. Per le dimensioni di questo corso MTZ basta.

3.8 Il protocollo del corso, dall'inizio alla fine

La sequenza di ogni esercizio

dati → modello → euristica e verifica → LP e duale
→ MIP → tabella → figure e notebook

```
m, x = modello(t, c, a)                                  # (1) dati, (2) modello

e = best_fit(t, a, lambda j, h, ra: c[j][h], "costo")    # (3) euristica
ub = sum(c[j][h] for (j, h) in e.x)
assert ammissibile(m, {f"x[{j},{h}]": 1 for (j, h) in e.x})   # vincoli, bound E interezza

```

```
d = duale(t, c, a)                                          # (4) duale scritto a mano
mano = {f"mu[{j}]": min(c[j]) for j in range(n)}
lb, viol = valuta(d, mano);  assert viol <= 1e-9
zlp, zlp_raff, pi = due_rilassamenti(m, d)                  # controlla la dualita' forte

z = risolvi(m)                                              # (5) il MIP
riga = registra_bound("assegnamento", ub, lb, zlp, zlp_raff, z)        # (6) la tabella
salva_dati(pd.DataFrame([riga]), "fam07_1_bound")           #      -> dati/fam07_1_bound.csv
```

Sull'istanza di assegnamento di questo capitolo il protocollo produce

$$LB = 10, \qquad z(LP) = z(LP^{+}) = \tfrac{53}{5}, \qquad z(MILP) = 11, \qquad UB = 11,$$

e la riga finisce in `dati/sched1_bound.csv`. Da lì la leggono la dispensa, la pagina del sito e `verifica_numeri.py`: **un solo posto in cui il numero esiste**.

I quattro numeri che ogni esercizio produce

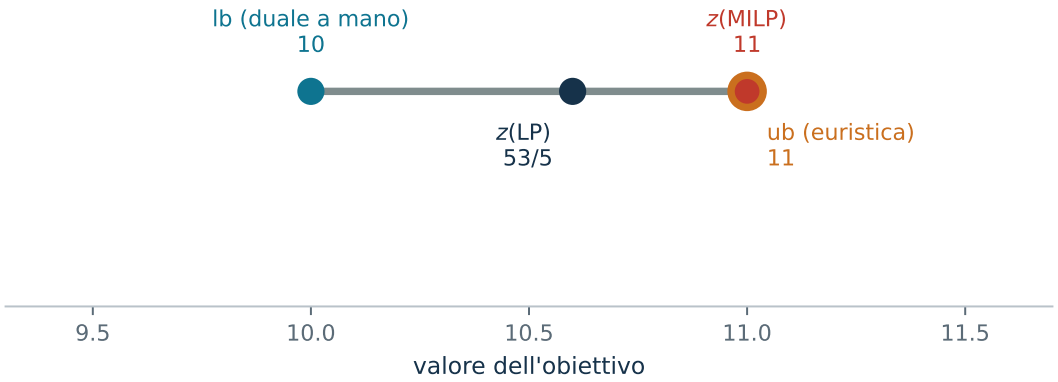


Figura 3.1: I quattro numeri del protocollo, sull'istanza di assegnamento.

3.9 Eseguire il codice

In locale

Dalla cartella `python/`:

```
python3 -m venv .venv && source .venv/bin/activate
pip install gurobipy pandas matplotlib
python3 fam07_1_assegnamento.py     # un capitolo
python3 esegui_tutti.py             # tutti, piu' i notebook
python3 verifica_numeri.py          # gli assert su ogni numero citato
```

Su Colab

Ogni capitolo ha un notebook che si apre dal badge sulla pagina del sito. La prima cella installa `gurobipy` e scarica i tre moduli comuni (`stile.py`, `mip.py`, `euristiche.py`) se non sono già presenti: sul computer non serve installare niente.

La licenza *size-limited* del pacchetto pip permette modelli fino a **2000 variabili e 2000 vincoli**. Le istanze di questo corso sono piccolissime — il modello più grande del corso ha poche decine di variabili — e ci stanno tutte con enorme margine. Per istanze più grandi si attiva gratuitamente la licenza accademica da portal.gurobi.com. Se un modello supera il limite, Gurobi lo segnala con un errore esplicito all'atto di `optimize()`: non produce silenziosamente un risultato sbagliato.

Capitolo 4

Euristiche costruttive

Script: `python/cap05_euristiche.py`, `python/euristiche.py` Pagina online

Un modello MILP si può risolvere all'ottimo, ma prima di risolverlo conviene avere in mano *una* soluzione ammissibile. Serve a due cose:

- dà un **bound primale**: un numero che l'ottimo non può superare (in un problema di minimo) o non può stare sotto (in un massimo);
- dà un punto di partenza al solver, e un termine di confronto per capire se il modello dice cose sensate.

Un'**euristica costruttiva** costruisce una soluzione aggiungendo un elemento per volta, senza mai tornare indietro. Non garantisce l'ottimo, e in generale non lo trova; e non garantisce nemmeno di arrivare a una soluzione ammissibile: può bloccarsi a metà, con un elemento che non entra da nessuna parte — proprio perché indietro non torna. Quello che garantisce è di fermarsi in fretta. Quando finisce con una soluzione ammissibile, il valore di quella soluzione è un bound primale; quando fallisce, bound primale non ce n'è.

Che cosa deve produrre un'euristica in questo corso

1. uno **pseudocodice** leggibile, con l'ordine di scansione, il criterio di scelta, la regola di spareggio e il caso di fallimento scritti esplicitamente;
2. la corrispondente **funzione Python**, riga per riga (in `python/euristiche.py`);
3. la **traccia** dell'esecuzione su un'istanza, che è il testo che compare nella dispensa;
4. il **controllo di ammissibilità** della soluzione prodotta: vincoli, domini *e* interezza;
5. il **bound** che ne esce, con il nome giusto: *UB* in un problema di minimo, *LB* in uno di massimo.

Il punto 4 non è una formalità. Una soluzione che soddisfa i vincoli lineari ma ha una componente frazionaria è ammissibile per il *rilassamento*, non per il MILP, e il suo valore non è un bound primale. La funzione `ammissibile` di `python/mip.py` controlla anche l'interezza, proprio per questa ragione.

I tre ingredienti

Ogni euristica costruttiva si descrive dicendo tre cose, e solo tre:

1. un **ordine** in cui si guardano gli elementi da collocare;
2. una **regola di inserimento**, che dice dove va l'elemento corrente;
3. un **controllo di ammissibilità**, che dice quando un inserimento non si può fare.

Cambiando uno solo dei tre si ottiene un'euristica diversa, con un valore diverso. Per questo, quando si presenta un'euristica, i tre ingredienti vanno scritti esplicitamente.

Un'euristica produce una soluzione ammissibile, e il suo valore è raggiungibile. In un problema di **minimo** quel valore è un bound *superiore*: $z(MILP) \leq UB$. In un problema di **massimo** è un bound *inferiore*: $LB \leq z(MILP)$. Chiamare UB il risultato di un'euristica costruttiva su un problema di massimo è l'errore di segno più frequente del corso.

4.1 Bin packing: le regole di inserimento

Il problema classico è il **bin packing**, il cui modello è nella sezione 3.1: degli oggetti vanno messi in contenitori tutti uguali, di capacità limitata, usandone il meno possibile. Qui non lo si risolve: lo si *costruisce*, una scelta per volta.

Next-fit, first-fit, best-fit

Input: n, pesi $w[1..n]$, capacità C, criterio γ
Output: i contenitori usati

Function *Costruisci*(n, w, C, γ)
  nessun contenitore aperto
  for $j := 1$ **to** n **do**
    `// next-fit:  solo il contenitore corrente, poi se ne apre uno nuovo`
    `// first-fit:  il primo contenitore aperto in cui l'oggetto ci sta`
    `// best-fit:  fra i contenitori in cui ci sta, quello di` $\gamma(j, b, res)$ `minimo`
    scegli b^* secondo la regola
    if *nessun contenitore aperto va bene* **then** aprine uno nuovo
    metti j in b^*; $res[b^*] \leftarrow res[b^*] - w[j]$
  end
  return i contenitori usati

Tutte e tre scandiscono gli oggetti *nell'ordine dato*: cambiare l'ordine cambia il risultato, e questo va detto quando si riporta un valore. I pareggi nel criterio γ si rompono sull'indice più piccolo, così l'esecuzione è riproducibile.

L'istanza. Sei oggetti di peso $w = (4, 4, 5, 3, 2, 3)$, in contenitori di capacità $C = 7$: sono gli stessi contenitori del modello della sezione 3.1, con due oggetti in più. L'istanza di là serve a scrivere il modello e sta piccola apposta — ma proprio per questo le tre regole vi rispondono tutte e tre «tre contenitori», e non si distinguono.

Euristica	come riempie	UB	$z(MILP)$	gap
next-fit	`[4]` `[4]` `[5]` `[3+2]` `[3]`	5	3	66,7%
first-fit	`[4+3]` `[4+2]` `[5]` `[3]`	4	3	33,3%
best-fit sul riempimento	`[4+3]` `[4+3]` `[5+2]`	3	3	0,0%

Il **bound elementare** chiude il discorso senza solver: il peso totale è $4+4+5+3+2+3 = 21$ e un contenitore ne porta 7, quindi servono almeno $\lceil 21/7 \rceil = 3$ contenitori. Il best-fit ne usa tre, e li riempie tutti e tre esatti: $LB = UB = 3$, e l'ottimo è *dimostrato* senza mai chiamare il solver.

Dove le tre regole si separano

Il **next-fit** chiude un contenitore appena un oggetto non ci sta, e non torna mai indietro: l'ultimo oggetto, di peso 3, entrerebbe esatto nel primo contenitore, che ha ancora 3 di spazio, ma quel contenitore non lo guarda più. Cinque contenitori invece di tre.
Fra **first-fit** e **best-fit** la differenza sta tutta sull'oggetto di peso 2. Il first-fit lo mette nel *primo* che lo accoglie, il secondo contenitore, che resta con 1 di spazio ormai inutile; il best-fit lo mette dove entra *esatto*, il terzo, che aveva 2. Così il 3 finale trova ancora posto nel secondo, e i contenitori restano tre.

4.2 $P||C_{\max}$: la regola del meno carico

Il secondo classico è lo **scheduling su macchine identiche**, visto anch'esso nella sezione 3.1, che in notazione standard si scrive $P||C_{\max}$: n lavori di durata t_j su k macchine uguali, minimizzando l'istante in cui finisce l'ultima. La regola naturale è il **list scheduling**: il lavoro corrente va sulla macchina in quel momento meno carica, e l'ordine in cui si guardano i lavori decide il risultato. L'ordine che funziona meglio è quello per **durata decrescente**, e la regola che ne esce ha un nome suo, **LPT** (*longest processing time first*).

L'idea

Per minimizzare il *makespan* — il carico della macchina più carica — su k macchine identiche: si ordinano i lavori per durata **decrescente** e si manda ciascuno sulla macchina di carico corrente minimo. L'ordine decrescente è essenziale: lasciare per ultimi i lavori lunghi li rende impossibili da sistemare.

LPT — longest processing time

Input: n, k, durate $t[1..n]$
Output: un'assegnazione x e il suo makespan

```
1 Function LPT(n, k, t)
2     L[m] ← 0 per ogni m                                  // carichi correnti
3     for j in ordine di t[j] decrescente do
4         m* ← arg min_m L[m]                  // pareggi: l'indice più piccolo
5         x[j][m*] ← 1;   L[m*] ← L[m*] + t[j]
6     end
7     return x, max_m L[m]
```

Sette lavori su tre macchine

$t = (5, 5, 4, 4, 3, 3, 3)$, $k = 3$, totale 27. La traccia:

- **Passi 1–3.** I lavori 5, 5, 4 vanno sulle tre macchine vuote: $L = (5, 5, 4)$.
- **Passo 4.** Lavoro 4: il carico minimo è la macchina 3, che passa a 8. $L = (5, 5, 8)$.
- **Passi 5–6.** I due lavori da 3 vanno sulle macchine 1 e 2: $L = (8, 8, 8)$.
- **Passo 7.** L'ultimo lavoro da 3 trova tutti i carichi pari a 8; per la regola dei pareggi va sulla macchina 1, che arriva a 11.

Makespan dell'LPT: $UB = 11$, con carichi $(11, 8, 8)$.
Il bound elementare. Il makespan è almeno $\max\left(\max_j t_j,\ \sum_j t_j/k\right) = \max(5,\ 27/3) = 9$: nessuna macchina può fare meno del lavoro più lungo, e almeno una deve portare la media. L'ottimo è proprio $z(MILP) = 9$ — si raggiunge con $\{5, 4\}$, $\{5, 4\}$, $\{3, 3, 3\}$ — e l'LPT sbaglia del 22,2%.

Due bound gratis, da confrontare

$\max_j t_j$ e $\sum_j t_j/k$ sono due bound inferiori che si calcolano senza risolvere niente, e il migliore dei due è già spesso vicino all'ottimo. Un bound «ovvio» che nessuno scrive è un bound sprecato: il duale del capitolo 2 serve quando quelli ovvi non bastano, non al loro posto.

4.3 Set covering: la regola del completamento più economico

Euristica costruttiva di copertura

Input: costi $c[1..n]$, insiemi $S[1..m]$ (chi copre la zona i)
Output: un insieme di elementi che copre tutte le zone

Function *EuristicaCopertura*(c, S)
  $scoperte \leftarrow \{1, 2, \ldots, m\}$; $y[j] \leftarrow 0$ per ogni j
  while $scoperte \neq \emptyset$ **do**
    per ogni j non ancora scelto: $nuove(j) \leftarrow |\{i \in scoperte : j \in S_i\}|$
    if $nuove(j) = 0$ *per ogni* j **then return** «nessuna soluzione trovata»
    $j^* \leftarrow \arg\min_{j\,:\,nuove(j)>0} c[j]/nuove(j)$
    $y[j^*] \leftarrow 1$; $scoperte \leftarrow scoperte \setminus \{i : j^* \in S_i\}$
  end
  return y

Il criterio è il **costo per zona nuova coperta**, non il costo assoluto: prendere l'elemento più economico senza guardare quanto copre porta a soluzioni molto peggiori.

Le quattro squadre del capitolo 2

Costi $c = (4, 3, 5, 3)$, sei zone (le sei coppie di quartieri).

- **Passo 1.** Tutte e sei le zone sono scoperte; i rapporti sono $4/3$, $3/3 = 1$, $5/3$, $3/3 = 1$; pareggio fra gli elementi 2 e 4, si sceglie il 2, che copre le zone 1, 2 e 5.
- **Passo 2.** Restano le zone 3, 4, 6; i rapporti sono $4/2 = 2$, $5/2$, $3/2$; si sceglie l'elemento 4, che copre le zone 4 e 6.
- **Passo 3.** Resta la zona 3; i rapporti sono $4/1$ e $5/1$; si sceglie l'elemento 1.

Soluzione $\{1, 2, 4\}$, di costo $UB = 10$, che qui è l'ottimo: $z(MILP) = 10$. Il bound duale costruito nel capitolo 2 vale 7: il gap certificato è del 30%, anche se l'euristica era già ottima.

4.4 Zaino: la regola del rapporto migliore

Lo **zaino** è il modello con cui si apre il capitolo 3: gli oggetti hanno un *valore* oltre a un peso, e la risorsa è una sola. La regola costruttiva naturale guarda il rapporto fra i due, e quello che produce è una soluzione ammissibile, quindi un bound primale.

Euristica costruttiva per rapporto valore/peso

Input: valori $p[1..n]$, pesi $w[1..n]$, capacità C
Output: un sottoinsieme ammissibile

```
Function EuristicaZaino(p, w, C)
    residuo ← C;   y[j] ← 0 per ogni j
    for j in ordine di p[j]/w[j] decrescente do
        if w[j] ≤ residuo then y[j] ← 1;   residuo ← residuo − w[j]
    end
    return y
```

Sull'istanza $p = (10, 7, 6, 4)$, $w = (5, 4, 3, 3)$, $C = 9$: i rapporti sono 2, 7/4, 2, 4/3; si prende l'oggetto 1 e poi l'oggetto 3 (peso 8), mentre il 2 e il 4 non entrano più. Valore 16. Poiché il problema è di *massimo*,

$$LB = 16 \;\leq\; z(MILP) = 17,$$

con un gap del 5,9%: l'ottimo prende gli oggetti 1 e 2, riempiendo lo zaino esattamente. L'euristica del rapporto sbaglia proprio perché l'oggetto 3, migliore come rapporto, lascia un residuo inutilizzabile.

4.5 TSP: il vicino più vicino

Il quarto classico è il **commesso viaggiatore** (*travelling salesman problem*, TSP), di cui il capitolo 3 ha dato le variabili: date n città e le distanze $d_{ij} \in \mathbb{Q}_{>0}$ fra ogni coppia, si cerca il giro più corto che le visiti tutte una volta sola e torni al punto di partenza. È il problema su cui la costruzione passo per passo si vede meglio, perché la soluzione è una sequenza: l'ordine *è* la soluzione.

La regola costruttiva classica è il **vicino più vicino** (*nearest neighbour*): si parte da una città, e ogni volta si va alla più vicina fra quelle non ancora visitate; quando non ne restano, si torna alla partenza. È ammissibile per costruzione — ogni città entra una volta sola, e l'ultimo arco chiude il giro — ed è veloce, perché a ogni passo guarda solo le distanze dalla città corrente.

Cinque città, cinque partenze

Le distanze, simmetriche:

	1	2	3	4	5
1	—	5	2	2	9
2	5	—	4	3	4
3	2	4	—	4	7
4	2	3	4	—	7
5	9	4	7	7	—

Partendo dalla città 1: la più vicina è la 3 (distanza 2); da lì la 2 (4); da lì la 4 (3); resta la 5 (7); e si torna alla 1, che costa 9. Il giro $1 \to 3 \to 2 \to 4 \to 5 \to 1$ è lungo 25.
L'ultimo arco è quello che si paga: la regola sceglie bene finché ha scelta, e all'ultimo passo non ne ha più. Cambiando la città di partenza cambia il giro, e cambia la lunghezza:

partenza	giro	lunghezza
1	$1 \to 3 \to 2 \to 4 \to 5 \to 1$	25
2	$2 \to 4 \to 1 \to 3 \to 5 \to 2$	18
3	$3 \to 1 \to 4 \to 2 \to 5 \to 3$	18
4	$4 \to 1 \to 3 \to 2 \to 5 \to 4$	19
5	$5 \to 2 \to 4 \to 1 \to 3 \to 5$	18

Con cinque città i giri distinti sono $(5-1)!/2 = 12$, e si possono enumerare tutti: l'ottimo è $1 \to 3 \to 5 \to 2 \to 4 \to 1$, lungo 18. Tre partenze su cinque lo trovano, una resta a 19 e quella da cui siamo partiti a 25, cioè il 38,9% sopra l'ottimo.
Due cose da portare via. La prima: l'euristica dà *una* soluzione ammissibile, quindi un bound superiore — qui $z(MILP) \le 25$ — e nient'altro; che 18 sia l'ottimo lo sappiamo per enumerazione, non perché l'euristica lo dica. La seconda: far girare la stessa regola da tutte le partenze e tenere il giro migliore costa n volte tanto e dà un bound superiore migliore. È la forma più semplice di *multi-start*, e non cambia la natura della cosa: resta un bound da un lato solo.

4.6 Lot sizing: copertura di periodi a costo unitario minimo

L'ultimo classico è il **lot sizing**: su T periodi, con domanda d_t da soddisfare, un costo fisso f ogni volta che si lancia una produzione e un costo h per ogni unità tenuta in magazzino per un periodo, si decide in quali periodi produrre e quanto. La tensione è tutta lì: lanciare di rado costa magazzino, lanciare spesso costa setup.

La regola costruttiva classica guarda proprio quel compromesso. Dal primo periodo con domanda si prova a coprire $1, 2, 3, \ldots$ periodi consecutivi, si calcola per ciascuna scelta il *costo medio per unità* prodotta — setup più magazzino, diviso la quantità — e si prende la copertura con il costo medio più basso; poi si riparte dal primo periodo rimasto scoperto.

Least unit cost

Input: domande $d[1..T]$, costo di lancio f, costo unitario di magazzino h
Output: i periodi in cui si lancia la produzione, e le quantità

Function *LeastUnitCost*(d, f, h)
  $t \leftarrow 1$
  while $t \le T$ **do**
    salta i periodi con $d[t] = 0$
    for $k := 1$ **to** $T - t + 1$ **do**
      $q_k \leftarrow \sum_{s=t}^{t+k-1} d[s]$; $c_k \leftarrow \big(f + h \sum_{s=t}^{t+k-1} (s-t)\, d[s]\big)/q_k$
    end
    $k^* \leftarrow \arg\min_k c_k$ `// il costo medio per unità più basso`
    produci q_{k^*} nel periodo t; $t \leftarrow t + k^*$
  end

Wagner–Whitin è un algoritmo **esatto** di programmazione dinamica per il modello di lot sizing *senza capacità*: risolve quel modello all'ottimo in tempo polinomiale. La procedura qui sopra è un'euristica, e il suo valore è solo un bound. Chiamarla «euristica costruttiva di Wagner–Whitin» confonde due cose diverse.

Cinque periodi

Domande $d = (20, 10, 30, 40, 10)$, lancio $f = 50$, magazzino $h = 1$.

- **Passo 1.** Dal periodo 1 conviene coprire 2 periodi (quantità 30, costo unitario 2).
- **Passo 2.** Dal periodo 3 conviene coprire 2 periodi (quantità 70, costo unitario $\approx$ 1,286).
- **Passo 3.** Dal periodo 5 resta solo quel periodo (quantità 10, costo unitario 5).

Costo della soluzione euristica: $UB = 200$. L'ottimo del MILP corrispondente è $z(MILP) = 170$, un gap del 17,6%: è anche il valore che darebbe Wagner–Whitin, essendo esatto su questo modello.

4.7 Ricerca locale, e che cosa non dà

Un passo oltre la costruzione

Una **ricerca locale** parte da una soluzione ammissibile e prova mosse elementari (sposta un lavoro, scambia due oggetti), accettando quelle che migliorano. Si ferma in un **ottimo locale**: una soluzione che nessuna mossa della famiglia scelta migliora.

Sulla soluzione LPT dell'esempio ($L = (11, 8, 8)$, makespan 11), la mossa «sposta un lavoro su un'altra macchina» non migliora nulla: spostare uno dei due lavori da 3 dalla macchina 1 porta il suo carico a 8 ma alza a 11 quello della macchina che lo riceve. La ricerca locale si ferma a 11, mentre l'ottimo è 9: per arrivarci serve uno *scambio* fra due macchine, non un semplice spostamento.

La ricerca locale restituisce una soluzione ammissibile, quindi un bound dal lato pessimistico, e nient'altro. Non dimostra niente sull'ottimo, non produce nessun bound dal lato ottimistico, e il fatto che si sia fermata non significa che sia arrivata. L'unico modo per sapere quanto sia buona è confrontarla con un bound dall'altro lato.

4.8 Quando un'euristica costruttiva fallisce

Un'euristica costruttiva può arrivare a un punto in cui l'elemento corrente non si può collocare da nessuna parte. Non è un errore di programmazione: è la conseguenza del non tornare mai indietro.

Tre lavori di durata $(3, 3, 2)$ su due macchine con disponibilità $(5, 3)$. Il next-fit: il lavoro 1 va sulla macchina 1 (residuo 2); il lavoro 2 non ci sta e passa alla macchina 2 (residuo 0); il lavoro 3 non ci sta e non ci sono altre macchine: **fallimento**. Ma il problema è ammissibile: i lavori 2 e 3 stanno insieme sulla macchina 1 ($3 + 2 = 5$) e il lavoro 1 sulla macchina 2 ($3 \leq 3$).
Un'euristica costruttiva è *miope*: decide una cosa alla volta e non torna indietro. Il suo fallimento è un'informazione sull'euristica, non sul problema. Per dimostrare che un modello è inammissibile serve il solver (`Status = INFEASIBLE`) o una dimostrazione, non un'euristica costruttiva andata male.

Se l'euristica non restituisce una soluzione, non si è ottenuto niente: non c'è alcun bound primale. Le due cose da fare sono, nell'ordine:

1. cambiare l'ordine — l'ordine per dimensione decrescente fallisce molto meno spesso di quello naturale, perché sistema per primi gli oggetti difficili;
2. cambiare la regola di inserimento.

Se anche così non si trova nulla, la domanda diventa se il problema sia ammissibile, e questa è una domanda per il modello, non per l'euristica.

4.9 Che cosa dà, e che cosa non dà

Il conto esatto di che cosa si sa

Sia $\bar{z}$ il valore della soluzione trovata da un'euristica costruttiva.

- In un problema di **minimo** si sa che
$$z(MILP) \leq \bar{z},$$
cioè $\bar{z}$ è un upper bound: $UB = \bar{z}$.
- In un problema di **massimo** si sa che
$$z(MILP) \geq \bar{z},$$
cioè $\bar{z}$ è un lower bound: $LB = \bar{z}$.
- In nessuno dei due casi si sa *quanto* $\bar{z}$ sia lontano dall'ottimo. Per saperlo serve un bound dall'altro lato, e quello non viene da una soluzione ammissibile: viene da un rilassamento o da un argomento combinatorio (capitolo 2).

Con entrambi i bound in mano si può scrivere il **gap certificato**: in assoluto $UB - LB$, e in percentuale
$$\frac{UB - LB}{|UB|}$$

in un minimo, dove UB è la soluzione euristica, e $(UB - LB)/|LB|$ in un massimo, dove a fare da incumbent è LB. Al denominatore sta sempre la soluzione che si ha in mano, mai $z(MILP)$: è questo che lo rende l'unica misura di qualità disponibile *prima* di risolvere il MILP.

Perché costruire l'euristica a mano, se c'è il solver

Un solver moderno trova soluzioni ammissibili da solo, spesso migliori. Costruire un'euristica a mano serve comunque, per tre ragioni:

- obbliga a capire che cosa rende una soluzione ammissibile, e questo è esattamente il contenuto dei vincoli;
- dà un valore di riferimento indipendente dal solver: se il solver restituisce di peggio, c'è un errore nel modello;
- nei problemi grandi il solver non arriva all'ottimo, e la qualità della soluzione finale dipende da quella iniziale.

4.10 Il quadro delle euristiche

Euristica	Verso	valore	$z(MILP)$	gap
next-fit (bin packing)	min (UB)	5	3	66,7%
first-fit (bin packing)	min (UB)	4	3	33,3%
best-fit sul riempimento (bin packing)	min (UB)	3	3	0,0%
LPT (makespan)	min (UB)	11	9	22,2%
euristica costruttiva di copertura	min (UB)	10	10	0,0%
euristica costruttiva per rapporto (zaino)	max (LB)	16	17	5,9%
nearest neighbour (TSP)	min (UB)	25	18	38,9%
least unit cost (lot sizing)	min (UB)	200	170	17,6%

Che cosa si impara da questa tabella

Due euristiche trovano l'ottimo e sei no, e *prima* di risolvere il MILP non c'è modo di sapere quali. Un gap del 0% e uno del 67% si distinguono soltanto *dopo*. È esattamente per questo che il corso chiede sempre due bound: un'euristica da sola dice quanto costa una soluzione che si può realizzare, non quanto si sta perdendo.

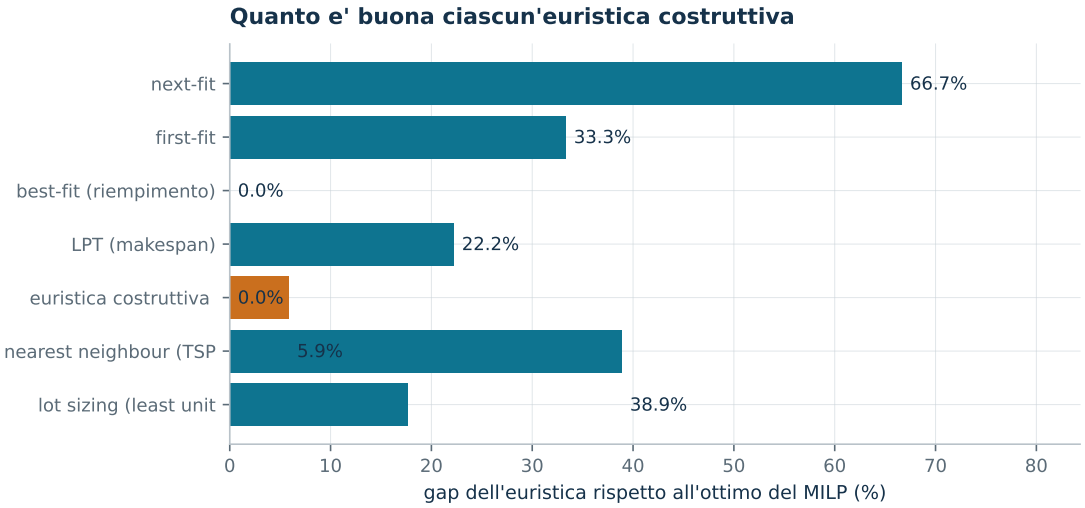


Figura 4.1: Il gap di ciascuna euristica rispetto all'ottimo del MILP, sulle istanze minime di questo capitolo. Petrolio per i problemi di minimo, arancione per quelli di massimo.

Capitolo 5

Logica e variabili binarie

Script: `python/cap02_logica.py`, `python/booleane.py` Pagina online

Una variabile binaria è una risposta «sì/no». Questo capitolo mostra come tradurre in *vincoli lineari* le condizioni logiche fra quelle risposte, e — soprattutto — come *dimostrare* che la traduzione è esatta.

5.1 Proposizioni e variabili binarie

Una proposizione, una variabile

Una **funzione booleana** restituisce uno dei due valori `TRUE` e `FALSE`; la si rappresenta con una variabile binaria

$$x \in \{0,1\}, \qquad x = 1 \iff \text{la proposizione è } \texttt{TRUE}, \qquad x = 0 \iff \text{è } \texttt{FALSE}.$$

Un'**espressione booleana** si costruisce con variabili binarie, i tre operatori `AND` (congiunzione, $\wedge$), `OR` (disgiunzione, $\vee$) e `NOT` (negazione, $\neg$), e le parentesi.

Due espressioni booleane

$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big),$$
$$(x_a\ \ \texttt{OR}\ \ x_b)\ \ \texttt{AND}\ \ (x_c\ \ \texttt{OR}\ \ x_d)\ \ \texttt{AND}\ \ \texttt{NOT}\ x_e.$$

Soddisfacibilità

Il **problema di soddisfacibilità booleana** chiede se esiste un'assegnazione di valori logici alle variabili che renda vera l'espressione. Se esiste, l'espressione è **soddisfacibile**; altrimenti è **insoddisfacibile**.

Soddisfacibili e no

- la formula
$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big)$$
è soddisfatta, per esempio, da $x_a = 0$ (le altre qualsiasi).

- la formula
$$(x_a \ \texttt{OR} \ x_b) \ \texttt{AND} \ (x_c \ \texttt{OR} \ x_d) \ \texttt{AND} \ \texttt{NOT} \ x_e$$
è soddisfatta da $x_a = 1$, $x_c = 1$, $x_b = x_d = x_e = 0$.
- $(x_a \ \texttt{OR} \ x_b) \ \texttt{AND} \ (\texttt{NOT} \ x_a \ \texttt{OR} \ x_b) \ \texttt{AND} \ (x_a \ \texttt{OR} \ \texttt{NOT} \ x_b) \ \texttt{AND} \ (\texttt{NOT} \ x_a \ \texttt{OR} \ \texttt{NOT} \ x_b)$ è insoddisfacibile: le quattro assegnazioni possibili di (x_a, x_b) falsificano, ciascuna, una delle quattro clausole.

5.2 Letterali, clausole, forma normale congiuntiva

Le tre parole

Un **letterale** è una variabile (x_a, letterale positivo) o la sua negazione ($\texttt{NOT} \ x_a$, letterale negativo). Una **clausola** è una disgiunzione (`OR`) di letterali, o un singolo letterale. Un'espressione è in **forma normale congiuntiva** (CNF) se è una congiunzione (`AND`) di clausole, o una singola clausola.

Delle due espressioni dell'esempio precedente, la seconda è in CNF e la prima no. Ogni espressione booleana *può* essere riscritta in CNF, usando le equivalenze che seguono.

Le equivalenze che servono

Per ogni terna di variabili binarie $x_a, x_b, x_c \in \{0, 1\}$:

$$x_a \ \texttt{AND} \ (x_b \ \texttt{OR} \ x_c) \iff (x_a \ \texttt{AND} \ x_b) \ \texttt{OR} \ (x_a \ \texttt{AND} \ x_c) \tag{5.1}$$
$$x_a \ \texttt{OR} \ (x_b \ \texttt{AND} \ x_c) \iff (x_a \ \texttt{OR} \ x_b) \ \texttt{AND} \ (x_a \ \texttt{OR} \ x_c) \tag{5.2}$$
$$\texttt{NOT} \, (x_a \ \texttt{OR} \ x_b) \iff \texttt{NOT} \ x_a \ \texttt{AND} \ \texttt{NOT} \ x_b \tag{5.3}$$
$$\texttt{NOT} \, (x_a \ \texttt{AND} \ x_b) \iff \texttt{NOT} \ x_a \ \texttt{OR} \ \texttt{NOT} \ x_b \tag{5.4}$$
$$x_a \ \texttt{OR} \ (x_a \ \texttt{AND} \ x_b) \iff x_a \tag{5.5}$$
$$x_a \ \texttt{AND} \ (x_a \ \texttt{OR} \ x_b) \iff x_a \tag{5.6}$$
$$\texttt{NOT} \, (\texttt{NOT} \ x_a) \iff x_a \tag{5.7}$$

Le (5.1)–(5.2) sono le **distributive**, le (5.3)–(5.4) sono le leggi di **De Morgan**, le (5.5)–(5.6) sono l'**assorbimento**. Valgono anche con più di due termini fra parentesi e con espressioni al posto delle variabili.

Ogni equivalenza si dimostra per casi: con due variabili ci sono 4 assegnazioni, con tre ce ne sono 8, e si verifica che i due membri hanno lo stesso valore su ciascuna. Lo script `cap02_logica.py` esegue questo controllo su tutte e sette le equivalenze.

Raccogliere: la (5.1) letta al contrario

Letta da destra a sinistra, la distributiva (5.1) permette di *raccogliere* due congiunzioni che condividono un letterale:

$$(x_a \ \texttt{AND} \ x_b) \ \texttt{OR} \ (x_a \ \texttt{AND} \ x_c) \iff x_a \ \texttt{AND} \ (x_b \ \texttt{OR} \ x_c).$$

Serve quando si porta in CNF una disgiunzione di più congiunzioni con letterali in comune:

riduce, passo dopo passo, il numero di termini da distribuire.

Due trasformazioni in CNF

- per la formula
$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big)$$
basta la (5.2), e diventa
$$(\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ x_d\ \ \texttt{OR}\ \ x_e).$$

- la formula
$$(x_a\ \ \texttt{AND}\ \ x_b)\ \ \texttt{OR}\ \ (x_a\ \ \texttt{AND}\ \ x_c)\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c)$$
dice «almeno due fra a, b, c». Si raccoglie x_a nelle prime due,
$$\big[x_a\ \ \texttt{AND}\ \ (x_b\ \ \texttt{OR}\ \ x_c)\big]\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c),$$
si applica la (5.2) e si ottengono i due fattori
$$x_a\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c)\qquad\text{e}\qquad(x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c);$$
il secondo si riduce a $(x_b\ \ \texttt{OR}\ \ x_c)$ per assorbimento (5.5), e il primo si distribuisce di nuovo. In tutto:
$$(x_a\ \ \texttt{OR}\ \ x_b)\ \ \texttt{AND}\ \ (x_a\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_b\ \ \texttt{OR}\ \ x_c).$$
Sarà la chiave dell'esercizio 2.4.

5.3 Da CNF a vincoli lineari

La traduzione, in tre regole

Un'espressione booleana in CNF è equivalente a un sistema di vincoli lineari sulle stesse variabili binarie:

1. ogni **clausola** diventa un vincolo di disuguaglianza ≥ 1;
2. ogni $\texttt{OR}$ dentro la clausola diventa un $+$;
3. ogni letterale negativo $\texttt{NOT}\ x$ diventa $1 - x$.

Perché è esatta

Una clausola con letterali positivi P e negativi N diventa
$$\sum_{i\in P} x_i + \sum_{i\in N}(1 - x_i) \geq 1 \qquad\Longleftrightarrow\qquad \sum_{i\in P} x_i - \sum_{i\in N} x_i \geq 1 - |N|.$$
Su una variabile binaria, $x_i = 1$ contribuisce 1 al membro sinistro se il letterale è positivo e soddisfatto, e $1 - x_i = 1$ contribuisce 1 se il letterale è negativo e soddisfatto. La somma conta *quanti letterali della clausola sono veri*: chiedere che sia ≥ 1 è esattamente chiedere

che la clausola sia vera. La congiunzione di clausole diventa il sistema di tutti i vincoli, perché tutte devono valere insieme.

Con due o più letterali negativi si preferisce, per leggibilità, la forma equivalente ottenuta moltiplicando per -1: per esempio $1 - x_1 + 1 - x_6 + x_7 \geq 1$ si scrive $x_1 + x_6 - x_7 \leq 1$. È lo stesso vincolo. Quello che non cambia mai è il numero di vincoli: *uno per clausola*, e le clausole si contano dopo la semplificazione (via le tautologie x `OR` `NOT` x e le clausole assorbite da una loro sottoclausola).

Tre traduzioni

$$(\texttt{NOT}\ x_a\ \texttt{OR}\ x_b\ \texttt{OR}\ x_c)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ x_d\ \texttt{OR}\ x_e) \iff \begin{cases} -x_a + x_b + x_c \geq 0, \\ -x_a + x_d + x_e \geq 0; \end{cases}$$

$$(x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (x_c\ \texttt{OR}\ x_d)\ \texttt{AND}\ \texttt{NOT}\ x_e \iff \begin{cases} x_a + x_b \geq 1, \\ x_c + x_d \geq 1, \\ x_e \leq 0; \end{cases}$$

$$(x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (x_a\ \texttt{OR}\ \texttt{NOT}\ x_b)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ \texttt{NOT}\ x_b)$$
$$\iff \begin{cases} x_a + x_b \geq 1, \\ -x_a + x_b \geq 0, \\ x_a - x_b \geq 0, \\ x_a + x_b \leq 1. \end{cases}$$

L'ultimo sistema è privo di soluzioni binarie, come l'espressione era insoddisfacibile: i primi tre vincoli danno $x_a = x_b = 1$, che viola il quarto.

5.4 Implicazioni logiche

L'implicazione fra due variabili

L'implicazione logica $x_a \Rightarrow x_b$ («se a allora b») equivale all'espressione booleana

$$\texttt{NOT}\ x_a\ \texttt{OR}\ x_b,$$

già in CNF, e quindi al vincolo lineare

$$1 - x_a + x_b \geq 1 \iff x_b - x_a \geq 0.$$

La contronominale è la stessa cosa

Per contrapposizione, $x_a \Rightarrow x_b$ implica anche la contronominale

$$\texttt{NOT}\ x_b \Rightarrow \texttt{NOT}\ x_a.$$

Non è un secondo vincolo: la sua espressione booleana è

$$\texttt{NOT}\,(\texttt{NOT}\ x_b)\quad \texttt{OR}\quad \texttt{NOT}\ x_a,$$

che per doppia negazione (5.7) è la stessa

$$\texttt{NOT}\ x_a\quad \texttt{OR}\quad x_b.$$

Un solo vincolo impone entrambe le letture — e nella descrizione di un modello conviene scrivere quella che si legge meglio nel contesto applicativo.

$x_b - x_a \geq 0$ dice «se a allora b», non «a se e solo se b»: ammette $x_a = 0$, $x_b = 1$. Per il «se e solo se» servono *due* implicazioni, cioè $x_a = x_b$, oppure una implicazione più un argomento di ottimalità, che è il modo in cui si scrive quando una delle due implicazioni segue dal segno del coefficiente nell'obiettivo.

La stessa ricetta vale quando antecedente e conseguente sono espressioni booleane qualsiasi: si scrive $\texttt{NOT}\,(\text{antecedente})\ \texttt{OR}\ (\text{conseguente})$, si porta in CNF e si traduce.

Le quattro combinazioni di base

Implicazione	Espressione in CNF	Vincoli	#
$x_a\ \texttt{AND}\ x_b \Rightarrow x_c$	$\texttt{NOT}\ x_a\ \texttt{OR}\ \texttt{NOT}\ x_b\ \texttt{OR}\ x_c$	$x_a + x_b - x_c \leq 1$	1
$x_a\ \texttt{OR}\ x_b \Rightarrow x_c$	$(\texttt{NOT}\ x_a\ \texttt{OR}\ x_c)\ \texttt{AND}\ (\texttt{NOT}\ x_b\ \texttt{OR}\ x_c)$	$x_c - x_a \geq 0,\ x_c - x_b \geq 0$	2
$x_a \Rightarrow x_b\ \texttt{AND}\ x_c$	$(\texttt{NOT}\ x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ x_c)$	$x_b - x_a \geq 0,\ x_c - x_a \geq 0$	2
$x_a \Rightarrow x_b\ \texttt{OR}\ x_c$	$\texttt{NOT}\ x_a\ \texttt{OR}\ x_b\ \texttt{OR}\ x_c$	$x_b + x_c - x_a \geq 0$	1

Si legga la tabella così: una *disgiunzione nell'antecedente* e una *congiunzione nel conseguente* costano due vincoli; una congiunzione nell'antecedente e una disgiunzione nel conseguente ne costano uno. È il motivo per cui il vincolo aggregato $\sum_j x_{jm} \leq k\, y_m$ e quello disaggregato $x_{jm} \leq y_m$ hanno costi diversi in numero di righe, e forze diverse nel rilassamento.

Un antecedente con tre letterali

$$x_a\ \texttt{AND}\ \texttt{NOT}\ x_b\ \texttt{AND}\ x_c \Rightarrow \texttt{NOT}\ x_d \iff \texttt{NOT}\ x_a\ \texttt{OR}\ x_b\ \texttt{OR}\ \texttt{NOT}\ x_c\ \texttt{OR}\ \texttt{NOT}\ x_d$$
$$\iff\quad x_a - x_b + x_c + x_d \leq 2.$$

5.5 Scindere un'implicazione: quando si può

Due scissioni valide e una no

$$(x_a \ \texttt{OR} \ x_b) \Rightarrow x_c \iff (x_a \Rightarrow x_c) \ \texttt{AND} \ (x_b \Rightarrow x_c) \tag{5.8}$$
$$x_a \Rightarrow (x_b \ \texttt{AND} \ x_c) \iff (x_a \Rightarrow x_b) \ \texttt{AND} \ (x_a \Rightarrow x_c) \tag{5.9}$$

La scissione analoga con l'antecedente **congiunzione** *non* è valida:

$$(x_a \ \texttt{AND} \ x_b) \Rightarrow x_c \quad \textbf{non} \iff \quad (x_a \Rightarrow x_c) \ \texttt{AND} \ (x_b \Rightarrow x_c).$$

Con $x_a = 1$, $x_b = 0$, $x_c = 0$ l'implicazione $(x_a \ \texttt{AND} \ x_b) \Rightarrow x_c$ è *vera* — il suo antecedente è `FALSE` — ma $x_a \Rightarrow x_c$ è *falsa*. La congiunzione delle due implicazioni scisse è quindi **strettamente più forte** dell'originale: taglia soluzioni che il problema ammette. Sbagliare qui non produce un modello «un po' diverso», produce un modello che può escludere l'ottimo.

Le due scissioni valide sono spesso la strada più corta: nella tabella precedente, le righe con due vincoli si ottengono direttamente da (5.8) o (5.9), senza passare da De Morgan e dalla distributiva.

5.6 Contare: al più uno, almeno uno, esattamente uno

Le condizioni di cardinalità su un insieme I di variabili binarie sono già lineari e non richiedono la CNF.

Condizione	Vincolo	Nota
almeno uno	$\sum_{i\in I} x_i \geq 1$	è la clausola $\bigvee_{i\in I} x_i$: *set covering*
al più uno	$\sum_{i\in I} x_i \leq 1$	*set packing*; equivale alle $\binom{\lvert I\rvert}{2}$ clausole $\texttt{NOT}\ x_i \ \texttt{OR} \ \texttt{NOT}\ x_j$, ma in un solo vincolo e con un rilassamento più stretto
esattamente uno	$\sum_{i\in I} x_i = 1$	*set partitioning*: le due precedenti insieme
almeno p	$\sum_{i\in I} x_i \geq p$	in CNF servirebbero $\binom{\lvert I\rvert}{\lvert I\rvert-p+1}$ clausole
al più p	$\sum_{i\in I} x_i \leq p$	in CNF servirebbero $\binom{\lvert I\rvert}{p+1}$ clausole
nessuno	$\sum_{i\in I} x_i \leq 0$	cioè $x_i = 0$ per ogni i

Un vincolo di cardinalità val più di molte clausole

«Al più uno fra tre» si può scrivere come tre clausole,

$$\texttt{NOT}\ x_1 \ \texttt{OR} \ \texttt{NOT}\ x_2, \qquad \texttt{NOT}\ x_1 \ \texttt{OR} \ \texttt{NOT}\ x_3, \qquad \texttt{NOT}\ x_2 \ \texttt{OR} \ \texttt{NOT}\ x_3,$$

cioè $x_1 + x_2 \leq 1$, $x_1 + x_3 \leq 1$, $x_2 + x_3 \leq 1$; oppure come il solo $x_1 + x_2 + x_3 \leq 1$. I due sistemi hanno le *stesse* soluzioni binarie, ma non lo stesso rilassamento: $x_1 = x_2 = x_3 = 1/2$ soddisfa le tre clausole e viola il vincolo aggregato. La versione compatta è più corta *e* più forte: un caso in cui non c'è niente da scambiare. Quando si può contare, si conta; la CNF serve quando la condizione non è una cardinalità.

5.7 Operatori logici e operatori insiemistici

Associando a ogni variabile binaria un sottoinsieme di un universo U — $x_a = 1$ significa «l'elemento scelto sta in A» — gli operatori logici diventano operatori insiemistici:

Logica	Insiemi	
x_a `AND` x_b	$A \cap B$	intersezione
x_a `OR` x_b	$A \cup B$	unione
`NOT` x_a	$\overline{A}$	complemento

Le equivalenze della sezione precedente diventano le identità insiemistiche $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$, $\overline{A \cap B} = \overline{A} \cup \overline{B}$, $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $A \cup (A \cap B) = A$ e $A \cap (A \cup B) = A$. Chi preferisce i diagrammi di Venn ha così una seconda dimostrazione delle stesse regole; chi preferisce l'enumerazione ha la prima. Le due si controllano a vicenda.

5.8 Verificare la traduzione, non fidarsene

Il criterio operativo

Una traduzione in vincoli lineari è corretta quando, per *ogni* assegnazione binaria delle variabili coinvolte, l'espressione è vera se e solo se tutti i vincoli sono soddisfatti. Con poche variabili questa verifica è una enumerazione di 2^n casi, che si fa in un istante: è la dimostrazione, per casi, del risultato. Il modulo `python/booleane.py` la esegue — e tutte le traduzioni della sezione seguente sono verificate così.

Il modulo fa anche il resto del lavoro del capitolo: elimina le implicazioni ($A \Rightarrow B$ diventa `NOT` A `OR` B), spinge le negazioni sui letterali con De Morgan, distribuisce l' `OR` sull' `AND` , scarta le clausole tautologiche e quelle assorbite, e infine applica le tre regole di traduzione. Non è un solutore SAT: è la trascrizione, riga per riga, delle regole di questo capitolo.

5.9 Cinque esercizi

In tutti e cinque, i progetti sono numerati e per ciascun progetto p la variabile binaria è

$$x_p = \begin{cases} 1 & \text{se il progetto } p \text{ è scelto,} \\ 0 & \text{altrimenti.} \end{cases}$$

Le soluzioni — espressione in CNF, vincoli lineari, conteggio dei vincoli e numero di assegnazioni su cui l'equivalenza è verificata per enumerazione — stanno nel documento delle soluzioni, riservato ai docenti.

Esercizio 5.1 — implicazioni dirette

Dieci progetti. Si modellino come vincoli lineari:

1. se si sceglie il progetto 2 allora si sceglie il 3;
2. se si sceglie il 2 allora non si sceglie il 4;
3. se si scelgono l'1 e il 6 allora si sceglie il 7;

4. se si sceglie l'1 oppure il 6 allora si sceglie l'8;
5. se si scelgono il 2 e il 3 allora non si sceglie il 9;
6. se si sceglie il 2 oppure il 3 allora non si sceglie il 10.

Esercizio 5.2 — antecedenti e conseguenti negati

Dieci progetti. Si modellino:

1. se non si sceglie il 3 allora si sceglie il 2;
2. se non si sceglie il 4 allora non si sceglie il 2;
3. se si sceglie il 7 allora si scelgono l'1 e il 6;
4. se si sceglie l'8 allora si sceglie l'1 oppure il 6;
5. se non si sceglie il 9 allora si scelgono il 2 e il 3;
6. se non si sceglie il 10 allora si sceglie il 2 oppure il 3.

Esercizio 5.3 — antecedenti e conseguenti composti

Otto progetti. Si modellino:

1. x_7 `OR` $x_3 \Rightarrow x_1$ `AND` x_2;
2. x_1 `AND` x_6 `AND` $x_7 \Rightarrow x_8$;
3. x_5 `AND` x_2 `AND` `NOT` $x_4 \Rightarrow$ `NOT` x_3;
4. (x_1 `OR` x_4) `AND` $x_6 \Rightarrow x_2$ `AND` (x_5 `OR` x_7);
5. (x_2 `OR` x_5) `AND` `NOT` $x_8 \Rightarrow x_3$ `OR` `NOT` x_6;
6. (x_1 `OR` x_4) `AND` (x_2 `OR` x_5) `AND` `NOT` $x_8 \Rightarrow x_3$ `AND` (`NOT` x_6 `OR` x_7).

Esercizio 5.4 — «almeno due fra»

Nove progetti. Si modellino:

1. se si sceglie il 4 allora almeno due fra 1, 2, 3;
2. se almeno due fra 6, 7, 8 allora si sceglie il 5;
3. se non si sceglie il 4 allora almeno due fra 1, 2, 3, 9;
4. se si sceglie l'8 allora (1 e 6) oppure (1 e 7) oppure (2 e 6);
5. se almeno due fra 1, 3, 5 allora non si sceglie il 9;
6. se (1 e 2) oppure (3 e 4) allora si sceglie il 5.

La condizione «almeno due fra 1, 2, 3» è anche $x_1 + x_2 + x_3 \geq 2$. Come conseguente dell'implicazione governata da x_4 diventa

$$x_1 + x_2 + x_3 \geq 2x_4 :$$

un vincolo solo, contro le tre clausole del punto 1. I due sistemi hanno le stesse 16 soluzioni binarie — si verifica per enumerazione — ma non lo stesso rilassamento, e questa volta è la forma contata a essere **più forte**. Sommando le tre clausole $x_i + x_j \geq x_4$ si ottiene infatti solo $x_1 + x_2 + x_3 \geq \frac{3}{2}x_4$, più debole di $\geq 2x_4$; viceversa, con $0 \leq x_i \leq 1$, la forma contata implica ciascuna clausola. Su max $x_1 + x_2 + x_3 + 3x_4$ con $x_1 + x_2 + x_3 + 2x_4 \leq 3$ e $z(MILP) = 3$, le tre clausole danno $z(LP^+) = 27/7 \approx 3{,}86$ e il vincolo contato $z(LP^+) = 15/4 = 3{,}75$: più vicino all'ottimo intero.
La morale non è «contare è sempre meglio»: nella sezione precedente «al più uno» contato era più forte delle clausole a coppie, qui «almeno due» contato è di nuovo più forte, ma non sempre va così — una somma aggregata può benissimo essere più *debole* delle disuguaglianze che riassume. La morale è che due formulazioni con lo stesso insieme intero vanno confrontate sul rilassamento, risolvendolo, e non a occhio.

Esercizio 5.5 — scissioni

Dieci progetti. Si modellino:

1. x_1 `OR` $x_2 \Rightarrow x_3$;
2. $x_4 \Rightarrow x_5$ `AND` x_6;
3. x_1 `OR` $x_2 \Rightarrow x_3$ `AND` x_4;
4. x_1 `AND` $x_2 \Rightarrow x_3$;
5. x_5 `OR` $x_6 \Rightarrow$ `NOT` x_7;
6. `NOT` x_8 `OR` `NOT` $x_9 \Rightarrow x_{10}$.

5.10 I vincoli logici dentro un modello di ottimizzazione

I vincoli di questo capitolo raramente vivono da soli: si aggiungono a un modello che ottimizza qualcosa. Con gli $n = 10$ progetti dell'esercizio 5.9, di ricavo r_p e costo c_p

progetto p	1	2	3	4	5	6	7	8	9	10
ricavo r_p	9	7	4	8	3	6	2	5	7	6
costo c_p	4	3	2	4	2	3	1	3	4	3

e budget $b = 14$, il modello è

Selezione di progetti con vincoli logici

$$\max \sum_{p=1}^{n} r_p\, x_p$$

$$\text{soggetto a} \sum_{p=1}^{n} c_p\, x_p \le b,$$

le clausole delle sei implicazioni,

$$x_p \in \{0,1\}, \qquad \forall p \in \{1,2,\ldots,n\}.$$

Descrizione. L'obiettivo somma i ricavi dei progetti scelti. Il primo vincolo, uno solo, è il budget. Le sei implicazioni, portate in forma normale congiuntiva, danno otto clausole, e ogni clausola è una disuguaglianza: non aggiungono variabili, solo righe. I domini sono le n condizioni di binarietà, una per progetto.

Il modello dell'istanza. Sui dati di sopra, una colonna per variabile:

Selezione di progetti, dieci binarie

$$\begin{aligned}
&\max\ 9x_1+7x_2+4x_3+8x_4+3x_5+6x_6+2x_7+5x_8+7x_9+6x_{10}\\
&\text{soggetto a } 4x_1+3x_2+2x_3+4x_4+2x_5+3x_6+x_7+3x_8+4x_9+3x_{10} \le 14\\
&-x_2+x_3 \ge 0\\
&x_2+x_4 \le 1\\
&x_1+x_6-x_7 \le 1\\
&-x_1+x_8 \ge 0\\
&-x_6+x_8 \ge 0\\
&x_2+x_3+x_9 \le 2\\
&x_2+x_{10} \le 1\\
&x_3+x_{10} \le 1\\
&x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10} \in \{0,1\}
\end{aligned}$$

Senza i vincoli logici l'ottimo è 30. Con i vincoli logici l'ottimo scende a $z(MILP) = 28$, ottenuto scegliendo i progetti $1, 2, 3, 5, 8$, di costo complessivo 14: il budget è saturo. Il rilassamento $z(LP^+)$ vale 29. Le sei implicazioni sono soddisfatte: si sceglie il 2 e infatti si sceglie anche il 3; non si sceglie il 4; non si scelgono insieme 1 e 6; si sceglie l'1 e infatti si sceglie l'8; si scelgono 2 e 3 e infatti non si sceglie il 9; si sceglie il 2 e infatti non si sceglie il 10.

Quanto tagliano sei implicazioni

Le $2^{10} = 1024$ assegnazioni delle dieci binarie si riducono a 234 quando si impongono tutte e sei le implicazioni dell'esercizio 5.9: meno di un quarto. La figura 5.1 mostra quante ne sopravvivono a ciascuna implicazione presa da sola e quante quando si accumulano. Nessuna di queste implicazioni, da sola, taglia più della metà dello spazio; insieme lo riducono di tre quarti. È il motivo per cui un modello si scrive un vincolo alla volta, ma si valuta tutto

intero.

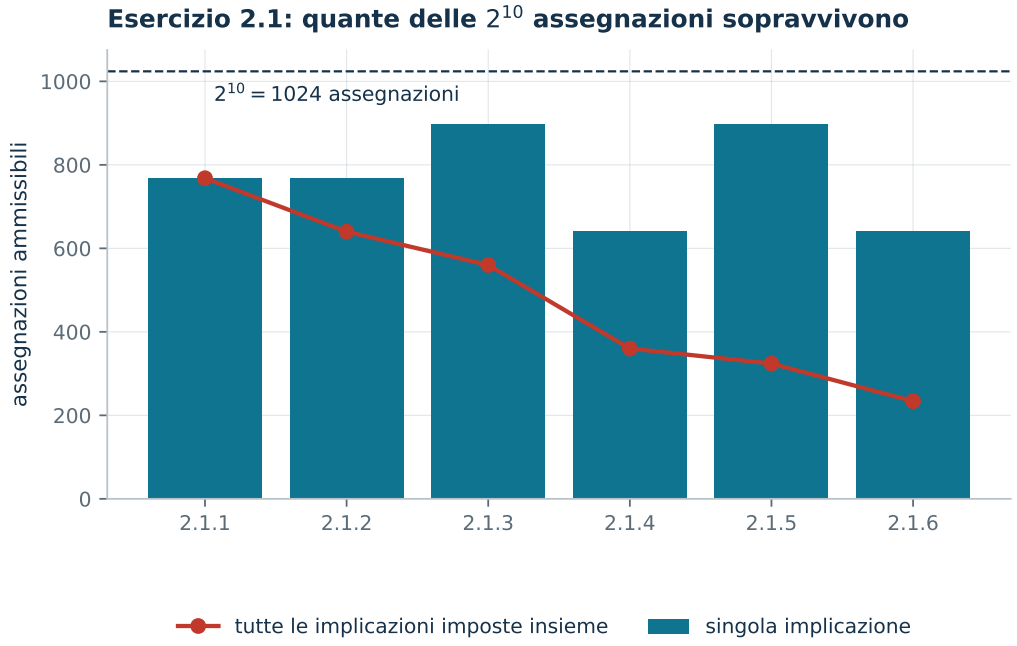


Figura 5.1: Esercizio 5.9: assegnazioni delle dieci binarie che sopravvivono a ciascuna implicazione presa da sola (barre) e a tutte quelle imposte fino a quel punto (linea).

Il modello in gurobipy

Nel codice che segue, accanto a `gurobipy`, compare il piccolo modulo didattico `booleane.py`, distribuito con il materiale del corso: serve solo a scrivere una formula logica come la si scriverebbe a mano — `V` crea una variabile, `AND`, `OR`, `NOT` e `IMP` la compongono — e a convertirla in forma normale congiuntiva con `cnf`. La funzione `vincolo` traduce poi una singola clausola nei coefficienti, nel verso e nel termine noto della disuguaglianza corrispondente. Tutto quello che fa è il procedimento di questo capitolo, scritto una volta sola.

```
from booleane import cnf, vincolo, IMP, AND, OR, NOT, V

x = {p: V(f"x{p}") for p in range(1, 11)}
implicazioni = [IMP(x[2], x[3]), IMP(x[2], NOT(x[4])),
                IMP(AND(x[1], x[6]), x[7]), IMP(OR(x[1], x[6]), x[8]),
                IMP(AND(x[2], x[3]), NOT(x[9])), IMP(OR(x[2], x[3]),
                   NOT(x[10]))]

m = gp.Model("selezione_progetti");  m.Params.OutputFlag = 0
xv = m.addVars(range(1, 11), vtype=GRB.BINARY, name="x")
m.setObjective(gp.quicksum(r[p] * xv[p] for p in range(1, 11)), GRB.
    MAXIMIZE)
m.addConstr(gp.quicksum(c[p] * xv[p] for p in range(1, 11)) <= b, name=
    "budget")
for i, formula in enumerate(implicazioni, 1):          # una clausola,
    un vincolo
```

```
    for j, clausola in enumerate(cnf(formula), 1):
        coef, verso, rhs = vincolo(clausola)
        lhs = gp.quicksum(k * xv[int(n[1:])] for n, k in coef.items())
        m.addConstr(lhs <= rhs if verso == "<=" else lhs >= rhs, name=f
            "logica{i}_{j}")
m.optimize()
```

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — riproducono in forma didattica i due lati del sandwich che il solver cerca progressivamente di chiudere.

5.11 Quello che questo capitolo ha messo in mano

Una condizione logica fra decisioni «sì/no» si scrive sempre allo stesso modo: si dà una binaria a ogni fatto elementare, si porta la condizione in forma normale congiuntiva — una congiunzione di OR — e ogni clausola diventa una disuguaglianza lineare. L'AND non costa nulla, perché i vincoli sono già in AND fra loro; l'OR diventa una somma ≥ 1; il NOT diventa $1 - x$; l'implicazione $x \Rightarrow y$ diventa $x \leq y$; il «se e solo se» chiede tutte e due le disuguaglianze.

Tre forme ricorrono così spesso da avere un nome: il *set covering* ($\sum_{j \in S} x_j \geq 1$, almeno uno), il *set packing* (≤ 1, al più uno) e il *set partitioning* ($= 1$, esattamente uno). Riconoscerle in un enunciato è metà del lavoro di modellazione.

Capitolo 6

Legami fra variabili

Script: `python/cap03_legami.py` Pagina online

Il capitolo 5 collega variabili *tutte binarie*. Qui si collegano famiglie *diverse*: binarie con continue, binarie con intere, continue fra loro. Sono quattordici tecniche, e sono il vero contenuto della modellazione MIP: quasi ogni modello di questo corso è una combinazione di queste.

Come si legge ogni sezione

Ogni tecnica è presentata in cinque passi:

(a) il **legame in parole**: che cosa si vuole dire;

(b) i **vincoli lineari** che lo dicono, con il conteggio;

(c) la **dimostrazione**: nei due versi se entrambi sono imposti dai vincoli; con un argomento di scambio se uno segue dall'ottimalità; con un *controesempio* quando una converso è falsa;

(d) la **forza del rilassamento**, su un'istanza minima risolta dallo script;

(e) la **riga** `gurobipy` che la scrive nel solver.

Il passo (c) è quello che distingue un modello corretto da uno plausibile. Il passo (d) è quello che distingue un modello che si risolve da uno che non si risolve.

Per tutto il capitolo distinguiamo:

- una proprietà **imposta dai vincoli**: vale per *ogni* soluzione ammissibile, ottima o no. Si dimostra guardando solo i vincoli;
- una proprietà **di ottimalità**: vale solo nelle soluzioni ottime. Si dimostra con un *argomento di scambio*: si prende una soluzione ammissibile che la viola, si costruisce una soluzione modificata, si verifica che resta ammissibile e si confronta il valore. Se il valore migliora *strettamente*, la conclusione è «in ogni ottimo»; se non peggiora, la conclusione è «esiste un ottimo», che è più debole.

Scrivere «in ogni ottimo» quando il coefficiente è solo non negativo è l'errore più comune di questo capitolo.

6.1 Attivazione: forma aggregata e forma disaggregata

(a) Il legame in parole. Una risorsa (una macchina, una sede, un impianto) va *attivata* prima di essere usata. Le binarie x_{ij} dicono «l'oggetto i usa la risorsa j», le binarie y_j dicono «la risorsa j è attivata». Si vuole: se qualcuno usa j, allora j è attivata.

(b) I vincoli. Due modi, con n oggetti e m risorse:

$$\text{disaggregato:} \quad x_{ij} \le y_j, \qquad \forall i \in \{1,2,\dots,n\},\ \forall j \in \{1,2,\dots,m\} \qquad (n\,m \text{ vincoli}), \tag{6.1}$$

$$\text{aggregato:} \quad \sum_{i=1}^{n} x_{ij} \le n\,y_j, \qquad \forall j \in \{1,2,\dots,m\} \qquad (m \text{ vincoli}). \tag{6.2}$$

Quando la risorsa ha una capacità k_j (al più k_j oggetti), la forma aggregata si scrive con quel coefficiente, $\sum_i x_{ij} \le k_j y_j$, e allora impone *anche* la capacità: due condizioni in un vincolo solo.

(c) La dimostrazione. Su punti binari le due forme sono **equivalenti**.

- *Disaggregato* $\Rightarrow$ *aggregato*: sommando (6.1) su i si ottiene $\sum_i x_{ij} \le n y_j$.
- *Aggregato* $\Rightarrow$ *disaggregato, su punti binari*: se $y_j = 0$, (6.2) dà $\sum_i x_{ij} \le 0$ e quindi $x_{ij} = 0 \le y_j$ per ogni i; se $y_j = 1$, allora $x_{ij} \le 1 = y_j$ per ogni i perché le x sono binarie.

Il secondo verso usa la binarietà di x e y: *fuori* dai punti binari non vale, ed è esattamente questo che rende le due forme diverse nel rilassamento.

Il verso opposto non è imposto. «Se j è attivata allora qualcuno la usa», cioè $y_j = 1 \Rightarrow \sum_i x_{ij} \ge 1$, **non** segue dai vincoli: $y_j = 1$ con tutte le $x_{ij} = 0$ è ammissibile in entrambe le forme. Segue dall'ottimalità quando attivare costa: se il costo di attivazione f_j è *strettamente* positivo, data una soluzione ottima con $y_j = 1$ e nessun $x_{ij} = 1$, porre $y_j = 0$ lascia tutti i vincoli soddisfatti e riduce il costo di $f_j > 0$ — contraddizione. Quindi «in ogni ottimo». Se invece $f_j = 0$ è ammesso, la conclusione corretta è la più debole «esiste un ottimo in cui $y_j = 0$».

(d) La forza del rilassamento. Istanza: $n = 3$ clienti, $m = 2$ sedi, costi di attivazione $f = (8, 6)$, costi di servizio

$$c = \begin{pmatrix} 2 & 5 \\ 4 & 1 \\ 3 & 3 \end{pmatrix}.$$

Formulazione	vincoli di link	$z(LP^+)$
aggregata (6.2)	$m = 2$	$38/3 \approx 12{,}67$
disaggregata (6.1)	$n\,m = 6$	15

L'ottimo intero è $z(MILP) = 15$: la forma disaggregata lo raggiunge già nel rilassamento, quella aggregata si ferma a $38/3$. Più righe, rilassamento più stretto: è il compromesso tipico di questa tecnica. Su istanze grandi la scelta non è ovvia — $n\,m$ righe possono essere molte — ma su istanze difficili la forma disaggregata quasi sempre paga.

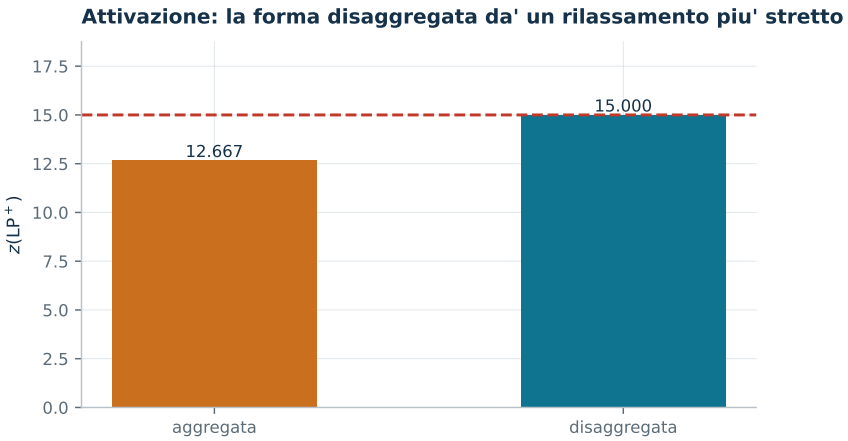


Figura 6.1: Il rilassamento delle due forme di attivazione sull'istanza a tre clienti e due sedi. La linea rossa è l'ottimo intero.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((x[i, j] <= y[j] for i in range(n) for j in range(mm)),
    name="link")    # disaggregato
m.addConstrs((x.sum("*", j) <= n * y[j] for j in range(mm)), name="link
    ")            # aggregato
```

6.2 Costo fisso, capacità e flusso continuo

(a) Il legame in parole. Come sopra, ma la quantità usata è *continua*: $q_j \geq 0$ è quanto si produce nell'impianto j, e y_j dice se l'impianto è aperto. Un impianto chiuso non produce; uno aperto produce al più la sua capacità c_j.

(b) I vincoli.

$$q_j \leq c_j\, y_j, \qquad \forall j \in \{1, 2, \ldots, m\} \qquad (m \text{ vincoli}). \tag{6.3}$$

Un solo vincolo per impianto, con il coefficiente *giusto*: la capacità, non un numero grande a caso.

(c) La dimostrazione. Se $y_j = 0$ il vincolo dà $q_j \leq 0$ e, con $q_j \geq 0$, forza $q_j = 0$: l'implicazione «chiuso $\Rightarrow$ non produce» è imposta dal vincolo, e con essa la sua contronominale «produce $\Rightarrow$ aperto». Se $y_j = 1$ il vincolo dà $q_j \leq c_j$: la capacità. Il verso «aperto $\Rightarrow$ produce» non è imposto e segue dall'ottimalità solo se $f_j > 0$, come nella sezione precedente.

(d) La forza del rilassamento. Istanza: due impianti, costi fissi $f = (10, 14)$, costi unitari $c = (3, 2)$, capacità $C = (6, 7)$, domanda $D = 9$. L'ottimo è $z(MILP) = 44$ (entrambi aperti, $q = (2, 7)$).

Coefficiente in (6.3)	$z(LP^+)$
la capacità, c_j	$112/3 \approx 37{,}33$
un big-M $= 100$ (più il vincolo $q_j \leq c_j$ a parte)	$1059/50 = 21{,}18$

Stesso insieme intero, stesso ottimo, rilassamenti lontanissimi. La regola: **il coefficiente di una binaria di attivazione è il più piccolo valore che la variabile continua può assumere quando l'attivazione vale 1**, e va ricavato dai dati.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="link")
```

6.3 Lotto minimo e variabile semicontinua

(a) Il legame in parole. «Se si produce, si produce almeno ℓ»: la quantità q_j vale zero oppure sta fra una soglia ℓ e la capacità c_j. Non è un intervallo: è l'unione di un punto e di un intervallo. Una variabile con questo dominio si chiama **semicontinua**; se la quantità è per di più intera — pezzi, unità, persone — si chiama **semi-intera**, e i due vincoli sono gli stessi.

(b) I vincoli. Con la stessa binaria y_j della sezione precedente,

$$\begin{aligned} \ell\, y_j &\le q_j, \qquad \forall j \in \{1,2,\dots,m\} \qquad (m \text{ vincoli}),\\ q_j &\le c_j\, y_j, \qquad \forall j \in \{1,2,\dots,m\} \qquad (m \text{ vincoli}). \end{aligned} \tag{6.4}$$

(c) La dimostrazione. Entrambi i versi sono imposti dai vincoli. Se $y_j = 0$: $0 \le q_j \le 0$, cioè $q_j = 0$. Se $y_j = 1$: $\ell \le q_j \le c_j$. Quindi

$$q_j \in \{0\} \cup [\ell,\ c_j],$$

esattamente il dominio voluto. Serve $\ell \le c_j$, altrimenti $y_j = 1$ è inammissibile e la variabile è costretta a zero: un errore di dati che il solver segnala come inammissibilità solo se y_j è forzata a 1 da altri vincoli.

(d) La forza del rilassamento. Sulla stessa istanza della sezione precedente con $\ell = 5$: l'ottimo passa da 44 a $z(MILP) = 49$, con $q = (5,5)$ invece di $(2,7)$ — la soglia costringe a produrre 5 nel secondo impianto anche se il primo è più caro. Ma $z(LP^+)$ resta $112/3$, identico al caso senza soglia.

> Nel rilassamento y_j è libera in $[0,1]$, e il vincolo $q_j \ge \ell y_j$ si soddisfa abbassando y_j: basta $y_j \le q_j/\ell$. Il vincolo di soglia *non morde mai* sul continuo. Tutto il suo effetto si scarica sull'interezza — ed è per questo che i modelli con lotto minimo sono tipicamente più difficili di quelli senza, a parità di dimensione.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((q[j] >= ell * y[j] for j in range(mm)), name="lotto")
m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="capacita")
```

Gurobi ha anche il tipo `GRB.SEMICONT`, che dichiara direttamente il dominio; in questo corso si scrive la formulazione a mano, perché è quella che si deve saper dimostrare.

6.4 Conteggi interi, capacità multipla, arrotondamento all'intero superiore

(a) Il legame in parole. «Quanti contenitori servono?» La variabile non è binaria ma *intera*: $w \in \mathbb{Z}_{\geq 0}$ conta oggetti indivisibili (scatole, camion, turni, operai) e ciascuno porta una capacità c.

(b) I vincoli.

$$\begin{aligned} \sum_i a_i\, x_i \;&\leq\; c\, w, && \text{(1 vincolo)},\\ w \;&\in\; \mathbb{Z}_{\geq 0}. && \text{(1 variabile intera)} \end{aligned} \tag{6.5}$$

(c) La dimostrazione. Posto $q = \sum_i a_i x_i$, il vincolo impone $w \geq q/c$ e l'interezza impone $w \geq \lceil q/c \rceil$. Insieme a un obiettivo che minimizza w (o che paga w), in ogni ottimo w vale esattamente quel tetto: se valesse di più, abbassarlo di 1 resterebbe ammissibile e ridurrebbe il costo. Con costo nullo per w la conclusione si indebolisce in «esiste un ottimo».

> $\lceil t \rceil$ non è una funzione lineare: non si può scrivere dentro un vincolo. La coppia «disuguaglianza $\leq cw$ + dichiarazione w intera» la realizza *implicitamente*, e va letta così quando si spiega il modello.

(d) La forza del rilassamento. 17 pezzi unitari, capienza $c = 5$: il rilassamento dà $w \geq 17/5 = 3{,}4$ e l'ottimo intero è $z(MILP) = 4$. Il gap $4 - 17/5 = 3/5$ viene tutto dall'interezza: nessun taglio lineare sulle sole x lo chiude, serve una disuguaglianza che usi w intera.

(e) In gurobipy, e dove si rivede.

```
w = m.addVar(vtype=GRB.INTEGER, name="w")
m.addConstr(gp.quicksum(a[i] * x[i] for i in range(n)) <= c * w, name="
    capienza")
```

6.5 L'ausiliaria di massimo

(a) Il legame in parole. Una variabile z deve valere il *massimo* di certe quantità: il tempo dell'attività più lunga, il costo di connessione più alto, il carico della macchina più carica.

(b) I vincoli. Se le quantità sono $t_j x_j$ con x_j binaria,

$$\begin{aligned} z \;&\geq\; t_j\, x_j, \quad \forall j \in \{1, 2, \ldots, n\} && (n \text{ vincoli}),\\ z \;&\geq\; 0. \end{aligned} \tag{6.6}$$

(c) La dimostrazione, in tre passi.

1. *Imposto dal vincolo*: per ogni j con $x_j = 1$ si ha $z \geq t_j$, quindi $z \geq \max_{j:x_j=1} t_j$. Vale in ogni soluzione ammissibile.

2. *Dall'ottimalità*: se z compare nell'obiettivo con coefficiente strettamente positivo in un minimo (o strettamente negativo in un massimo) e in nessun altro vincolo, allora in ogni ottimo z assume il valore più piccolo ammissibile. Infatti, data una soluzione ottima con $z > \max_{j:x_j=1} t_j$, abbassare z fino a quel massimo lascia tutti i vincoli (6.6) soddisfatti e migliora strettamente l'obiettivo.

3. *Sintesi*: in ogni ottimo, $z = \max_{j:x_j=1} t_j$ esattamente (con la convenzione che il massimo sull'insieme vuoto è 0, garantita da $z \geq 0$).

> Se z compare in un altro vincolo che la vuole *grande* — per esempio un secondo vincolo $z \geq w$, con w un'altra variabile — l'argomento di scambio non funziona più: abbassare z può violare quel vincolo. In quel caso $z \geq \max$ resta vero, ma $z = \max$ no. È l'errore tipico quando la stessa ausiliaria si riusa per due scopi: se serve il massimo di due famiglie diverse, servono due ausiliarie.

(d) La forza del rilassamento. $\min z$ con $z \geq t_j x_j$, $t = (4, 7, 3)$ e $\sum_j x_j \geq 2$: l'ottimo intero è $z(MILP) = 4$ (si scelgono i lavori 1 e 3, e il massimo è 4). Il rilassamento vale $168/61 \approx 2{,}75$: la soluzione frazionaria $x_j = z/t_j$ spalma la scelta su tutti e tre i lavori e abbassa il massimo. Il legame di massimo dà rilassamenti *deboli*: è una delle ragioni per cui i problemi di makespan sono difficili.

(e) In gurobipy, e dove si rivede.

```
z = m.addVar(name="z")
m.addConstrs((z >= t[j] * x[j] for j in range(n)), name="massimo")
```

6.6 Min-max, max-min e differenza fra il massimo e il minimo

(a) Il legame in parole. Tre obiettivi di equità, spesso confusi: *min-max* (minimizzare il carico più alto), *max-min* (massimizzare il carico più basso) e *differenza* (minimizzare lo scarto fra il più alto e il più basso).

(b) I vincoli. Con ℓ_k il carico della risorsa k ($k \in \{1, 2, \ldots, m\}$):

$$\text{min-max:} \quad \min z \quad \text{con} \quad z \geq \ell_k,\ \forall k \in \{1, 2, \ldots, m\} \qquad (m \text{ vincoli}), \tag{6.7}$$

$$\text{max-min:} \quad \max u \quad \text{con} \quad u \leq \ell_k,\ \forall k \in \{1, 2, \ldots, m\} \qquad (m \text{ vincoli}), \tag{6.8}$$

$$\text{differenza:} \quad \min (z - u) \quad \text{con entrambi} \qquad (2m \text{ vincoli}). \tag{6.9}$$

(c) La dimostrazione. Ciascuna è l'ausiliaria di massimo (o di minimo) della sezione precedente, con l'argomento di scambio nel verso giusto: in un $\min z$ la z scende al massimo dei carichi; in un $\max u$ la u sale al minimo dei carichi. Nella differenza entrambe le spinte sono presenti contemporaneamente e le due conclusioni valgono insieme.

> Sull'istanza a cinque pesi $p = (3, 5, 2, 4, 7)$ da ripartire su due operai (totale 21), le tre versioni scelgono la *stessa* ripartizione $(11, 10)$ — la migliore possibile, perché il totale è

dispari — ma i loro valori ottimi sono 11, 10 e 1. Sono tre numeri diversi che descrivono la stessa soluzione. Confrontare «$z = 11$» di un min-max con «$z = 1$» di una differenza non significa niente: prima di confrontare due modelli bisogna che ottimizzino la stessa cosa. E le soluzioni ottime possono anche non coincidere: con più di due risorse, min-max e max-min in generale scelgono ripartizioni diverse.

(d) La forza del rilassamento. Il min-max su quell'istanza dà $z(LP^+) = 21/2 = 10{,}5$ contro $z(MILP) = 11$: il rilassamento distribuisce i pesi a metà esatta, cosa che l'interezza non permette.

(e) In gurobipy, e dove si rivede.

```
z = m.addVar(name="z")
m.addConstrs((z >= carico[k] for k in range(len(carico))), name="max")
m.setObjective(z, GRB.MINIMIZE)
```

6.7 Il valore assoluto

(a) Il legame in parole. Si vuole $|u - v|$: uno scarto, un errore, uno sbilanciamento. Il caso in obiettivo e il caso in vincolo si comportano in modo radicalmente diverso.

(b) I vincoli.

- **In obiettivo, minimizzando**: una variabile $d \geq 0$ e due vincoli,

$$\begin{aligned} d &\geq u - v, && \text{(1 vincolo)}, \\ d &\geq v - u, && \text{(1 vincolo)}, \end{aligned} \tag{6.10}$$

  con d nell'obiettivo da minimizzare. Nessuna binaria.

- **Come vincolo** $\leq$: $|u - v| \leq k$ è semplicemente $u - v \leq k$ e $v - u \leq k$ (2 vincoli, nessuna binaria).

- **Come vincolo** $\geq$: $|u - v| \geq k$ **non** si scrive senza binarie. È la disgiunzione «$u - v \geq k$ *oppure* $v - u \geq k$», e richiede una binaria b e un big-M:

$$\begin{aligned} u - v &\geq k - M(1 - b), && \text{(1 vincolo)}, \\ v - u &\geq k - M\,b. && \text{(1 vincolo, 1 binaria)} \end{aligned} \tag{6.11}$$

(c) La dimostrazione. Nel primo caso i due vincoli (6.10) impongono $d \geq |u - v|$ (uno dei due membri destri *è* $|u-v|$); l'obiettivo, che minimizza d e in cui d non compare altrove, la porta all'uguaglianza con l'argomento di scambio della sezione 6.5. Nel terzo caso, $b = 1$ disattiva il secondo vincolo (se $M \geq k + \max(v - u)$) e lascia il primo, e viceversa: è una disgiunzione, non una congiunzione, e senza binaria si imporrebbero entrambi — cioè $0 \geq 2k$, inammissibile per $k > 0$.

$|u - v| \le k$ è l'intersezione di due semipiani: un insieme convesso, che si scrive con due vincoli lineari. $|u - v| \ge k$ è il complemento di una striscia: *non* è convesso, e nessun sistema di vincoli lineari senza variabili intere può descriverlo. La binaria non è un trucco: è necessaria.

(d) La forza del rilassamento. Sull'istanza dei cinque pesi, il problema min $|\ell_1 - \ell_2|$ ha ottimo $z(MILP) = 1$ e rilassamento $z(LP^+) = 0$: il continuo divide 21 in due metà uguali e azzera lo scarto. Il rilassamento di un obiettivo di valore assoluto è tipicamente 0, cioè inutile.

(e) In gurobipy, e dove si rivede.

```
d = m.addVar(name="d")
m.addConstr(d >= u - v, name="abs_piu")
m.addConstr(d >= v - u, name="abs_meno")
```

La stessa idea torna nella sezione 6.13, in una forma equivalente con due deviazioni.

6.8 Big-M: vincoli condizionati e disgiunzioni

(a) Il legame in parole. «Se $y = 1$ allora vale il vincolo $a'x \le b$; se $y = 0$ il vincolo non c'è». Un vincolo che si accende e si spegne.

(b) I vincoli.

$$a'x \;\le\; b + M\,(1 - y) \qquad (1 \text{ vincolo}). \tag{6.12}$$

(c) La dimostrazione, e come si sceglie M. Con $y = 1$ il vincolo è $a'x \le b$: acceso. Con $y = 0$ diventa $a'x \le b + M$: spento *purché* M sia abbastanza grande da non tagliare nulla, cioè

$$M \;\ge\; \max\{a'x : x \text{ ammissibile per gli altri vincoli}\} - b. \tag{6.13}$$

Si distinguono tre cose:

- un M **valido**: soddisfa (6.13), quindi il modello ha l'insieme intero giusto;
- un M **migliorabile**: valido ma più grande del necessario; l'insieme intero è lo stesso, il rilassamento è più debole;
- il **minimo** M **dimostrato**: il più piccolo valore per cui si è *dimostrata* la validità, tipicamente calcolando il massimo in (6.13) sui soli bound delle variabili. Non è necessariamente il minimo assoluto, ed è onesto dirlo.

Con $a = (3, 4, 5)$, $b = 6$ e x binarie, il massimo di $a'x$ è 12, quindi $M = 12 - 6 = 6$ è valido. Su max $x_1 + x_2 + x_3 + y$ l'ottimo è $z(MILP) = 3$ (con $y = 0$ e tutte le x a 1). Con $M = 5$ il vincolo con $y = 0$ resta $3x_1 + 4x_2 + 5x_3 \le 11$, che esclude $x = (1, 1, 1)$: l'ottimo scende a 2. Il modello non risponde più alla domanda posta.

(d) La forza del rilassamento. Sulla stessa istanza:

M	$z(LP^+)$	
6 (il minimo dimostrato)	3	coincide con $z(MILP)$
20	$37/10 = 3{,}7$	
1000	$1997/500 \approx 3{,}994$	quasi il massimo possibile, 4

Il degrado è monotono e rapido. La regola operativa: **calcolare M dai dati, sempre, e scriverlo nel testo subito dopo il modello**.

(e) In gurobipy, e dove si rivede.

```
M = sum(max(a[j], 0) for j in range(n)) - b        # calcolato dai dati, non scelto a occhio
m.addConstr(gp.quicksum(a[j] * x[j] for j in range(n)) <= b + M * (1 - y), name="cond")
```

La stessa idea torna nella sezione 6.9.

6.9 Precedenze e sequenziamento

(a) Il legame in parole. Su una macchina che esegue un lavoro alla volta, per ogni coppia di lavori uno dei due precede l'altro. È una disgiunzione per coppia, non una precedenza fissata dai dati.

(b) I vincoli. Con $\kappa_j \geq 0$ l'istante di completamento del lavoro j, t_j la sua durata e $s_{ij} \in \{0,1\}$ che vale 1 se j precede i:

$$s_{ij} + s_{ji} = 1, \qquad \forall i,j \in \{1,2,\dots,n\},\ i<j \qquad (n(n-1)/2 \text{ vincoli}), \tag{6.14}$$

$$\kappa_i \geq \kappa_j + t_i - M\,(1 - s_{ij}), \qquad \forall i,j \in \{1,2,\dots,n\},\ i \neq j \qquad (n(n-1) \text{ vincoli}), \tag{6.15}$$

$$\kappa_j \geq t_j, \qquad \forall j \in \{1,2,\dots,n\} \qquad (n \text{ vincoli}). \tag{6.16}$$

(c) La dimostrazione, e il minimo M. Il vincolo (6.14) impone che esattamente uno dei due ordini sia scelto. Se $s_{ij} = 1$, (6.15) diventa $\kappa_i \geq \kappa_j + t_i$: il lavoro i finisce almeno t_i dopo la fine di j, cioè non si sovrappongono. Se $s_{ij} = 0$ diventa $\kappa_i \geq \kappa_j + t_i - M$, che deve essere sempre soddisfatto. Poiché $\kappa_i \geq t_i$ e $\kappa_j \leq \sum_h t_h$ in ogni soluzione sensata, basta

$$M \;=\; \sum_{h=1}^{n} t_h,$$

perché allora $\kappa_j + t_i - M \leq \sum_h t_h + t_i - \sum_h t_h = t_i \leq \kappa_i$. L'orizzonte temporale $\sum_h t_h$ va *dichiarato*: senza un limite superiore sui κ_j, nessun M finito è valido.

(d) La forza del rilassamento. Tre lavori di durata $(3,2,4)$ su una macchina, obiettivo makespan: $z(MILP) = 9 = \sum_h t_h$ (ovvio: una sola macchina), con completamenti $(3,5,9)$. Il rilassamento vale 4: con $s_{ij} = 1/2$ tutti i vincoli di precedenza si spengono a metà e i lavori possono sovrapporsi. È il rilassamento più debole di tutto il capitolo, e spiega perché i modelli di sequenziamento big-M scalano male.

(e) In gurobipy, e dove si rivede.

```
M = sum(t)                                          # l'orizzonte, dichiarato
for i in range(n):
    for j in range(i):
        m.addConstr(s[i, j] + s[j, i] == 1, name=f"disg{i}{j}")
        m.addConstr(kappa[i] >= kappa[j] + t[i] - M * (1 - s[i, j]),
            name=f"prec{i}{j}")
        m.addConstr(kappa[j] >= kappa[i] + t[j] - M * (1 - s[j, i]),
            name=f"prec{j}{i}")
```

6.10 «Se e solo se»

(a) Il legame in parole. Una binaria y deve valere 1 *esattamente quando* una condizione è vera — per esempio «tutti i lavori della classe sono eseguiti». Non basta un'implicazione: ne servono due.

(b) I vincoli. Con $x_1, \dots, x_p$ binarie e la condizione «tutte valgono 1»:

$$y \le x_j, \qquad \forall j \in \{1, 2, \dots, p\} \qquad (p \text{ vincoli}), \tag{6.17}$$

$$y \ge \sum_{j=1}^{p} x_j - (p-1) \qquad (1 \text{ vincolo}). \tag{6.18}$$

(c) La dimostrazione, nei due versi.

- (6.17) dà $y = 1 \Rightarrow x_j = 1$ per ogni j; equivalentemente, se anche uno solo degli x_j è zero allora $y = 0$.
- (6.18) dà il verso opposto: se tutti gli $x_j = 1$, il membro destro vale $p - (p-1) = 1$, quindi $y \ge 1$ e dunque $y = 1$. Se almeno uno è zero, il membro destro è ≤ 0 e il vincolo non dice nulla.

Insieme, i due impongono $y = 1 \iff \sum_j x_j = p$ in *ogni* soluzione ammissibile.

Quando basta un verso solo. Se y compare nell'obiettivo di un massimo con un premio $v > 0$ e in nessun altro vincolo, il verso (6.18) si può omettere: in ogni ottimo, se tutti gli $x_j = 1$ e $y = 0$, alzare y a 1 resta ammissibile per (6.17) e aumenta l'obiettivo di $v > 0$. È un argomento di ottimalità, e vale solo all'ottimo.

Sull'istanza a tre lavori, ricavi $(2, 2, 2)$, capacità 3:

premio v	vincoli	$z(MILP)$	y, x all'ottimo	y dice «classe completa»?
9	solo (6.17)	15	$y = 1$, $x = (1, 1, 1)$	sì
9	entrambi	15	$y = 1$, $x = (1, 1, 1)$	sì
0	solo (6.17)	6	$y = 0$, $x = (1, 1, 1)$	**no**
0	entrambi	6	$y = 1$, $x = (1, 1, 1)$	sì

Con $v = 0$ il valore ottimo è lo stesso nei due modelli, ma nel terzo caso y non è più un indicatore fedele. Se y serve solo a pagare un premio, il verso mancante è inutile; se y

compare in *altri* vincoli — e questo è il caso non appena si aggiunge una condizione sul numero di classi complete — va scritto.

(d) La forza del rilassamento. $z(LP^+) = 15 = z(MILP)$ su questa istanza: entrambi i versi imposti danno un rilassamento esatto. Il vincolo (6.18) è debole nel rilassamento (con $x_j = 1/2$ il membro destro è negativo), ma qui non serve.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((y <= x[j] for j in range(p)), name="iff_su")
m.addConstr(y >= gp.quicksum(x[j] for j in range(p)) - (p - 1), name="
    iff_giu")
```

6.11 Contare i tipi diversi

(a) Il legame in parole. «Si devono produrre almeno due tipi diversi», «al più tre alimenti in dieta», «non più di quattro configurazioni». Si conta *quanti* elementi sono attivi, non quanto se ne produce.

(b) I vincoli. Con $q_j \geq 0$ la quantità e y_j l'indicatore:

$$q_j \leq c_j\, y_j, \qquad \forall j \in \{1, 2, \ldots, n\} \qquad (n \text{ vincoli}), \tag{6.19}$$

$$q_j \geq \ell\, y_j, \qquad \forall j \in \{1, 2, \ldots, n\} \qquad (n \text{ vincoli}), \tag{6.20}$$

$$\sum_{j=1}^{n} y_j \geq p \qquad (1 \text{ vincolo}). \tag{6.21}$$

(c) La dimostrazione, e perché servono entrambe le soglie. (6.19) dà «$y_j = 0 \Rightarrow q_j = 0$»; (6.20) dà il verso mancante «$y_j = 1 \Rightarrow q_j \geq \ell$». Senza (6.20), una soluzione con $y_j = 1$ e $q_j = 0$ è ammissibile: il vincolo di conteggio (6.21) si soddisfa accendendo indicatori vuoti, e la condizione «almeno p tipi diversi» non dice più niente. **Il conteggio dei tipi funziona solo se ogni tipo acceso produce davvero qualcosa**, e la soglia ℓ è ciò che lo garantisce.

(d) La forza del rilassamento. Tre tipi, ricavi unitari $(4, 3, 5)$, risorsa 12, capacità 10 ciascuno, soglia $\ell = 3$, almeno due tipi: $z(MILP) = 57$, con $q = (3, 0, 9)$. Il rilassamento vale anch'esso 57: qui il conteggio non introduce alcun gap.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((q[j] <= c[j] * y[j] for j in range(n)), name="attiva")
m.addConstrs((q[j] >= ell * y[j] for j in range(n)), name="lotto")
m.addConstr(y.sum() >= p, name="almeno_p_tipi")
```

6.12 Alldiff ed espansione binaria

(a) Il legame in parole. Due esigenze diverse ma imparentate: dare a n oggetti n valori *tutti distinti* (*alldiff*), e rappresentare una variabile intera limitata con variabili binarie (*espansione binaria*).

(b) I vincoli. *Alldiff*, con $p_{iv} = 1$ se l'oggetto i riceve il valore v:

$$\sum_{v=1}^{n} p_{iv} = 1, \qquad \forall i \in \{1, 2, \ldots, n\} \qquad (n \text{ vincoli}), \tag{6.22}$$

$$\sum_{i=1}^{n} p_{iv} = 1, \qquad \forall v \in \{1, 2, \ldots, n\} \qquad (n \text{ vincoli}). \tag{6.23}$$

Espansione binaria di $v \in \{0, 1, \ldots, 2^p - 1\}$:

$$\begin{aligned} v &= \sum_{k=0}^{p-1} 2^k \, b_k, && (1 \text{ vincolo}), \\ b_k &\in \{0, 1\}, \qquad \forall k \in \{0, 1, \ldots, p-1\} && (p \text{ binarie}). \end{aligned} \tag{6.24}$$

(c) La dimostrazione. L'alldiff è un doppio *set partitioning*: il primo gruppo dà «ogni oggetto un valore», il secondo «ogni valore a un solo oggetto». Insieme impongono una biiezione, cioè valori tutti distinti. L'espansione binaria è la rappresentazione in base 2, unica per ogni intero in quell'intervallo: la corrispondenza fra v e $(b_0, \ldots, b_{p-1})$ è biunivoca.

L'alldiff ha rilassamento esatto, l'espansione no

La matrice del doppio partitioning è quella del problema di assegnamento: è totalmente unimodulare, quindi tutti i vertici del rilassamento sono interi e $z(LP^+) = z(MILP)$. Sull'istanza 3×3 dello script entrambi valgono 7: l'interezza è gratis. L'espansione binaria, al contrario, non aggiunge forza: $\sum_k 2^k b_k$ con $b_k \in [0, 1]$ copre tutto $[0, 2^p - 1]$ in modo continuo, esattamente come $v \geq 0$, $v \leq 2^p - 1$. Serve a *riformulare*, non a rafforzare — per esempio quando un'altra parte del modello ha bisogno di indicatori binari e non di una variabile intera.

(d) L'esempio. $v \in \{0, \ldots, 7\}$ con $v \geq 5$, $\min v$: il modello dà $v = 5 = 1+4$, cioè $(b_0, b_1, b_2) = (1, 0, 1)$.

(e) In gurobipy, e dove si rivede.

```
m.addConstrs((p.sum(i, "*") == 1 for i in range(n)), name="un_valore")
m.addConstrs((p.sum("*", v) == 1 for v in range(n)), name="alldiff")
m.addConstr(v == gp.quicksum(2 ** k * b[k] for k in range(p)), name="
    espansione")
```

6.13 Vincoli violabili, deviazioni e penalità

(a) Il legame in parole. Un vincolo che si *preferirebbe* rispettare, ma che può essere violato pagando: la domanda di un periodo, una preferenza di orario, un obiettivo di servizio. Trasformare un vincolo rigido in uno morbido è spesso la differenza fra un modello inammissibile e uno utile.

(b) I vincoli. Al posto di $a'x = \beta$ si scrive

$$\begin{aligned} a'x + s^- - s^+ &= \beta, && \text{(1 vincolo)}, \\ s^- &\geq 0, && \text{(1 variabile continua)}, \\ s^+ &\geq 0, && \text{(1 variabile continua)}, \end{aligned} \tag{6.25}$$

e nell'obiettivo si aggiunge $\pi^- s^- + \pi^+ s^+$ con penalità $\pi^-, \pi^+ > 0$. La variabile s^- misura di quanto si sta *sotto* il target, s^+ di quanto si sta *sopra*.

(c) La dimostrazione. Il vincolo, da solo, è sempre soddisfacibile: qualunque $a'x$ si può compensare con una delle due deviazioni. La proprietà interessante è che **in ogni ottimo almeno una delle due deviazioni è nulla**: se $s^- = \sigma > 0$ e $s^+ = \tau > 0$, sottrarre $\min(\sigma, \tau)$ da entrambe lascia (6.25) soddisfatto (la differenza $s^- - s^+$ non cambia) e riduce l'obiettivo di $(\pi^- + \pi^+)\min(\sigma, \tau) > 0$. Quindi s^- e s^+ sono davvero le parti positiva e negativa dello scarto, e $s^- + s^+ = |a'x - \beta|$ in ogni ottimo: la stessa cosa della sezione 6.7, scritta con un'uguaglianza invece che con due disuguaglianze.

> L'argomento di scambio richiede solo $\pi^- + \pi^+ > 0$. Con $\pi^- = 0$ e $\pi^+ > 0$ ogni ottimo ha ancora almeno una deviazione nulla, ma restare sotto β non costa niente: il vincolo soft non spinge più $a'x$ verso l'obiettivo. Solo se $\pi^- + \pi^+ \leq 0$ una soluzione ottima può avere entrambe le deviazioni positive (con somma negativa il modello è addirittura illimitato), e la lettura «s^- è la sotto-copertura» si perde. Vale la stessa regola di tutto il capitolo: la forza della conclusione dipende dal segno dei coefficienti nell'obiettivo.

(d) L'esempio. Domanda 6 in ciascuno di tre periodi, disponibilità totale $15 < 18$, penalità $\pi^+ = 3$ e $\pi^- = 2$. Il modello rigido sarebbe inammissibile; quello morbido dà $z(MILP) = 6$, con $q = (3, 6, 6)$ e una sotto-copertura di 3 concentrata sul primo periodo. Nulla, nei dati, dice che vada concentrata: qualunque ripartizione della sotto-copertura totale 3 ha lo stesso costo, e il solver ne restituisce una.

(e) In gurobipy, e dove si rivede.

```
sm = m.addVars(T, name="s_meno");  sp = m.addVars(T, name="s_piu")
m.addConstrs((q[t] + sm[t] - sp[t] == domanda[t] for t in range(T)),
    name="target")
m.setObjective(costo + gp.quicksum(pen_giu * sm[t] + pen_su * sp[t] for
    t in range(T)),
                GRB.MINIMIZE)
```

6.14 Funzioni lineari a tratti

(a) Il legame in parole. Il costo non è proporzionale alla quantità: cambia a scaglioni — sconti sopra una soglia, sovrapprezzi oltre una capacità, tariffe a fasce. La funzione $g(q)$ è continua e lineare a tratti, con nodi $q_0 < q_1 < \cdots < q_K$ e valori $g_0, \ldots, g_K$.

(b) I vincoli. Con $\lambda_k \geq 0$ i pesi della combinazione convessa e $w_t \in \{0,1\}$ l'indicatore del tratto t (fra q_{t-1} e q_t):

$$\sum_{k=0}^{K} \lambda_k = 1, \tag{6.26}$$

$$q = \sum_{k=0}^{K} q_k \lambda_k, \qquad \text{(1 vincolo)}, \tag{6.27}$$

$$g(q) = \sum_{k=0}^{K} g_k \lambda_k, \qquad \text{(1 vincolo)}, \tag{6.28}$$

$$\sum_{t=1}^{K} w_t = 1, \qquad \text{(1 vincolo)}, \tag{6.29}$$

$$\lambda_k \leq \sum_{t\,:\,k\in\{t-1,t\}} w_t, \qquad \forall k \in \{0,1,\dots,K\} \qquad (K+1 \text{ vincoli}). \tag{6.30}$$

(c) La dimostrazione. Il vincolo (6.30) è l'**adiacenza**: λ_k può essere positiva solo se il tratto scelto ha q_k come estremo. Con (6.29) il tratto è uno solo, quindi al più due λ consecutive sono positive, e allora $(q, g(q))$ è un punto del segmento fra (q_{t-1}, g_{t-1}) e (q_t, g_t): cioè un punto *del grafico*. Senza l'adiacenza si potrebbero mescolare nodi non consecutivi, e (q, g) finirebbe nell'**inviluppo convesso inferiore** del grafico — sotto la funzione.

Quando g è convessa e si *minimizza*, l'inviluppo convesso inferiore coincide con la funzione e le λ ottime risultano automaticamente adiacenti: il modello senza binarie è già corretto. Quando g non è convessa — il caso degli sconti di quantità, in cui il costo marginale *scende* — la differenza si vede subito. Con nodi $(0, 4, 10, 16)$ e valori $(0, 12, 30, 36)$ e domanda $q \geq 13$:

Formulazione	valore ottimo
senza adiacenza (combinazione convessa libera)	$117/4 = 29{,}25$
con adiacenza (6.30)	33

La prima mescola i nodi 0 e 3 (le uniche λ non nulle) e restituisce un costo che la funzione *non assume in nessun punto*: è il valore dell'inviluppo, non del grafico. La seconda mescola i nodi 2 e 3, adiacenti, e dà il valore esatto $g(13) = 30 + 6 \cdot \frac{3}{6} = 33$.

(d) La forza del rilassamento. Attenzione a non confondere due cose: l'adiacenza cambia l'*insieme intero* (i due modelli hanno ottimi diversi, 29,25 e 33), ma non la forza del rilassamento — con w_t frazionarie il vincolo (6.30) non morde e i due modelli hanno lo stesso $z(LP^+) = 117/4$.

(e) In gurobipy, e dove si rivede.

```
m.addConstr(lam.sum() == 1, name="convessa")
m.addConstr(q == gp.quicksum(nodi[k] * lam[k] for k in range(K + 1)),
    name="ascissa")
m.addConstr(w.sum() == 1, name="un_tratto")
```

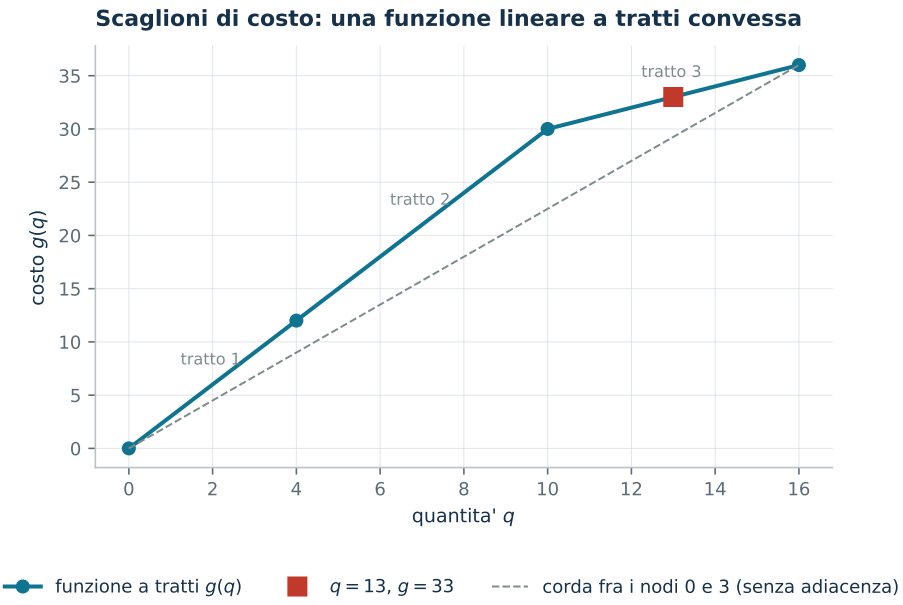


Figura 6.2: La funzione a tratti con sconto di quantità. La corda tratteggiata fra i nodi estremi è ciò che la formulazione senza adiacenza permette di raggiungere: sta *sotto* il grafico.

```
for k in range(K + 1):                          # adiacenza: lambda_k solo sui tratti che toccano k
    m.addConstr(lam[k] <= gp.quicksum(w[t] for t in (k - 1, k) if 0 <= t < K),
                name=f"adiacenza{k}")
```

Gurobi offre anche i vincoli `addGenConstrPWL` e i tipi SOS2, che fanno lo stesso lavoro internamente; qui la formulazione manuale resta il materiale principale, perché è quella di cui si deve saper dimostrare la correttezza.

6.15 La mappa delle tecniche

La tabella seguente è pensata per la consultazione: si legge dalla prima colonna («che cosa voglio dire») alla seconda («come si scrive»), e la terza ricorda che cosa va dichiarato ogni volta che la si usa.

Tecnica	Formulazione	Da dichiarare
6.1 attivazione	$x_{ij} \le y_j$ (disagg.) oppure $\sum_i x_{ij} \le k_j y_j$ (agg.)	quale forma, e perché; segno di f_j per il verso di ottimalità
6.2 costo fisso	$q_j \le c_j y_j$	che c_j è la capacità, non un big-M
6.3 lotto minimo	$\ell y_j \le q_j \le c_j y_j$	che $\ell \le c_j$; che il rilassamento non ne risente
6.4 conteggio intero	$\sum_i a_i x_i \le K w$, w intera	che l'interezza realizza il tetto
6.5 ausiliaria di massimo	$z \ge t_j x_j$, $z \ge 0$	che z non compare altrove; segno nell'obiettivo
6.6 min-max / max-min	$T \ge L_k$ e $\min T$; $U \le L_k$ e $\max U$	quale dei tre obiettivi, e che non si confrontano
6.7 valore assoluto	$d \ge \pm(u - v)$ in obiettivo; disgiunzione se $\ge k$	se è obiettivo o vincolo, e in quale verso
6.8 big-M	$a'x \le b + M(1 - y)$	il valore di M calcolato dai dati
6.9 precedenze	$s_{ij}+s_{ji} = 1$, $\kappa_i \ge \kappa_j + t_i - M(1-s_{ij})$	l'orizzonte e $M = \sum_h t_h$
6.10 se e solo se	$y \le x_j$ e $y \ge \sum_j x_j - (p-1)$	se il secondo verso serve o segue dall'ottimalità
6.11 conteggio dei tipi	$\ell y_j \le q_j \le c_j y_j$, $\sum_j y_j \ge p$	che senza la soglia ℓ il conteggio è vuoto
6.12 alldiff / espansione	doppio partitioning; $v = \sum_k 2^k b_k$	che l'alldiff ha rilassamento esatto
6.13 vincoli violabili	$a'x + s^- - s^+ = \beta$ con penalità	i due segni delle penalità
6.14 funzione a tratti	combinazione convessa + adiacenza	se g è convessa; altrimenti l'adiacenza è obbligatoria

Le tre domande da farsi davanti a un legame nuovo

1. **Quale verso è imposto dai vincoli e quale no?** Si risponde guardando solo i vincoli, un caso per valore della binaria.
2. **Il verso mancante segue dall'ottimalità?** Si risponde con l'argomento di scambio, e la risposta dipende dal *segno* del coefficiente nell'obiettivo: strettamente diverso da zero dà «in ogni ottimo», nullo dà al più «esiste un ottimo».
3. **Quanto è forte il rilassamento?** Si risponde confrontando $z(LP^+)$ con $z(MILP)$ su un'istanza piccola, e — quando ci sono due formulazioni — confrontandole fra loro. Le due formulazioni vanno prima dimostrate equivalenti sui punti interi: senza quel passaggio non si stanno confrontando due modelli dello stesso problema.

Indice analitico

argomento di scambio, 60, 65, 66, 72, 75
attivazione, 21, 29, 61, 63

big-M, 66, 68
bin packing, 25, 38
bound duale, 9, 10, 14, 19, 41
bound primale, 10, 37, 45

commesso viaggiatore, 25, 42

disuguaglianze valide, 23
dualità debole, 8
dualità forte, 9, 14, 17

espansione binaria, 70
euristica costruttiva, 17, 19, 37, 42, 44, 45

funzione lineare a tratti, 72

gap certificato, 41
gap di integralità, 45

lot sizing, 43
lotto minimo, 63
LPT, 39

ricerca locale, 44
rilassamento lineare, 6, 8

scheduling su macchine identiche, 39
set covering, 10, 15, 59
set packing, 10, 59
set partitioning, 10, 59

tolleranze, 9

valore assoluto, 9, 67
variabile semi-intera, 63
variabile semicontinua, 63
vicino più vicino, 42

zaino, 24, 41